СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ 2

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 3

ВВЕДЕНИЕ 6

1 АНАЛИЗ ЛИТЕРАТУРЫ 9
1.1 Структура и организация научных и исследовательских задач . 9
1.1.1 Научный процесс как пайплайн с обратной связью 9
1.1.2 Основные этапы и артефакты научной задачи 10
1.1.3 Проблемы организации научного процесса 11
1.1.4 Вывод 11
1.2 Инструменты автоматизации научных задач 12
1.2.1 Инструменты частичной автоматизации 12
1.2.2 Системы автоматизации экспериментов и scientific workflows 13
1.2.3 Инструменты на базе искусственного интеллекта 14
1.2.4 Сравнительный анализ и ограничения существующих подходов 14
1.2.5 Выводы 15
1.3 Автоматизация научных задач агентными системами 16
1.3.1 От одиночных агентов к мультиагентным системам 17
1.3.2 Современные архитектуры LLM-агентов и их ограничения 18
1.3.3 Агентные системы для научного процесса 19
1.3.4 Архитектурные подходы к построению научных агентных систем 21
1.3.5 Сравнение архитектурных подходов 22
1.3.6 Выводы 24
1.4 Фреймворк OpenClaw и система ClawdLab 25
1.4.1 OpenClaw как инфраструктурная основа 25
1.4.2 ClawdLab как лабораторная надстройка над OpenClaw . 26
1.4.3 Текущая организация процесса в ClawdLab 28

1.4.4 Сильные стороны и ограничения OpenClaw/ClawdLab . 29
1.4.5 Связь с дальнейшим образом решения TO BE 31
1.5 Образ целевого решения TO BE и архитектурная гипотеза 32
1.5.1 Переход от AS IS к TO BE 32
1.5.2 Структурированная память исследования 33
1.5.3 Циклическая модель выполнения задач 34
1.5.4 Границы целевого решения 35
1.5.5 Ожидаемые эффекты от целевого решения 36
1.5.6 Архитектурная гипотеза 36
1.5.7 Визуализация AS IS и TO BE архитектур 37

2 ДИЗАЙН ЭКСПЕРИМЕНТОВ 40
2.1 Постановка экспериментальной задачи 40
2.1.1 Типы научных задач, поддерживаемые ClawdLab 40
2.1.2 Границы эксперимента 41
2.1.3 Исследовательский вопрос и ожидаемые эффекты 42
2.2 Обзор инструментов, технологий и методов для экспериментальной реализации 42
2.2.1 Требования к выбираемым технологиям 43
2.2.2 Технологии структурированной памяти 43
2.2.3 Технологии циклического выполнения задач 46
2.2.4 Инструменты оркестрации и реализации 49
2.2.5 Выбор подходов для экспериментальной реализации ... 50
2.3 Экспериментальные данные и задачи 51
2.3.1 Выбор типов задач для оценки 51
2.3.2 Задачи анализа научной информации 52
2.3.3 Задачи глубокого исследования 52
2.3.4 Подготовка входных данных и эталонных результатов . 53
2.4 Экспериментальные конфигурации 54
2.4.1 Базовая конфигурация ClawdLab 54
2.4.2 Конфигурация со структурированной памятью 54
2.4.3 Конфигурация с циклическим процессом 55
2.4.4 Итоговая конфигурация с обеими доработками 55
2.4.5 Абляционное сравнение 56
2.5 Методика оценки 56

2.5.1 Общая логика валидации ... 56
2.5.2 Метрики аналитика ... 57
2.5.3 Метрики синтезатора ... 58
2.5.4 Оценка артефактов и трассируемости ... 58
2.5.5 Метрики цикличности и использования критики ... 59
2.5.6 LLM-судья (LLM-as-a-Judge) ... 60
2.5.7 Метрики воспроизводимости и изоляции прогонов ... 60
2.5.8 Метрики вычислительной эффективности ... 61
2.5.9 Критерии принятия гипотезы ... 61
2.5.10 Протокол проведения экспериментов ... 63
2.6 Ожидаемый проверочный прототип ... 64
2.6.1 Целевая архитектура экспериментального прототипа ... 64
2.6.2 Ожидаемые сценарии работы ... 66
2.6.3 Возможности дальнейшего развития прототипа ... 67

3 ОПИСАНИЕ ПРЕДЛАГАЕМОГО РЕШЕНИЯ ... 69
3.1 Архитектура продакшн-решения и System Design ... 69
3.1.1 Цели и требования к целевой архитектуре ... 69
3.1.2 Логическая структура решения ... 70
3.1.3 Физическое развертывание на VPS-сервере ... 70
3.2 Разработка сервиса и MVP-интеграция ... 71
3.2.1 Базовый контур MVP ... 71
3.2.2 Доработка literature provider ... 71
3.2.3 Архитектура модуля структурированной памяти ... 72
3.2.4 Архитектура доработки цикличности исполнения ... 75
3.3 Развертывание, DevOps и MLOps-практики ... 76
3.3.1 Организация развертывания и запуск сервисов ... 76
3.4 Тестирование и верификация решения ... 76
3.4.1 План тестирования и валидации ... 76

4 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА ... 77
4.1 Реализация доработки со структурированной памятью ... 77
4.2 Итоговое сравнение метрик между конфигурациями ... 77

ЗАКЛЮЧЕНИЕ ... 78

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ …… 78

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

ФИО: Есаулова Наталья Андреевна Тема ВКР: ”Разработка мультиагентной конфигурации для оптимизации решения исследовательских и научных задач на базе OpenClaw” Задание ВКР:

– Проведение анализа подходов к применению мультиагентных систем для решения научных и исследовательских задач.
– Проведение анализа архитектуры и функциональных возможностей OpenClaw.
– Проектирование расширенной мультиагентной конфигурации OpenClaw, направленной на повышение эффективности решения научных задач.
– Реализация модулей и механизмов их взаимодействия в данной конфигурации.
– Разработка MVP-сервиса для тестирования и демонстрации работы системы.
– Проведение тестирования системы и оценка ее эффективности.

Аннотация ВКР:

Цель исследования

Целью работы является разработка и проверка архитектурной доработки агентной платформы на базе OpenClaw агентов ClawdLab для повышения эффективности выполнения научных задач.

Задачи, решаемые в ВКР

В работе решаются следующие задачи: анализ предметной области и современного состояния агентных систем для научных задач; анализ архитектуры OpenClaw и ClawdLab, их возможностей и ограничений; формулирование архитектурной гипотезы о применении структурированной памяти и управляемой цикличности выполнения задач; проектирование экспериментальной схемы для сравнения базовой и улучшенной конфигураций; разработка MVP-контура ClawdLab на собственной инфраструктуре; реализация доработкок; оценка результатов в различных конфигурациях системы; подведение итогов.

Краткая характеристика полученных результатов

В рамках работы был проведен обзор существующих подходов к применению мультиагентных LLM-систем в научных задачах и выделены архитектурные ограничения ClawdLab, связанные с потерей контекста, недостаточной трассируемос артефактов и неформальным характером цикличности выполнения задач. Был спроектирован экспериментальный контур для проверки доработок на задачах анализа и глубокого исследования в рамках определенного исследовательского вопроса. Также был развернут MVP-сервис ClawdLab и агенты OpenClaw и

выполнены необходимые доработки системы для ее тестирования.

Основным практическим результатом стала реализация двух доработок: структурирс памяти и управляемой цикличности выполнения задач. Для памяти была предложена и реализована трехуровневая модель, включающая рабочий, эпизодичес и канонический уровни, а также сущности для хранения записей памяти, ссылок на источники и продвижения объектов памяти между уровнями. Для цикличности была спроектирована и добавлена логика фиксированных правил повторного выполнения задач, позволяющая связывать новую попытку с критикой, причиной повторного запуска и условиями остановки. Вместе эти доработки позволяют сохранять происхождение выводов, связывать результаты задач с источниками и артефактами, а также делать процесс выполнения научных задач более прозрачным и воспроизводимым.

ВВЕДЕНИЕ

В последние годы наблюдается устойчивый рост интереса к агентным системам, в частности к многоагентным архитектурам на базе больших языковых моделей. Это связано с переходом от статических моделей обработки данных к более гибким системам, способным автономно планировать, декомпозировать задачи и взаимодействовать с внешними инструментами. Особенно заметен рост числа работ, посвященных применению таких систем в исследовательских задачах: автоматизация обзоров литературы, генерация гипотез, постановка и проведение вычислительных экспериментов. Однако, несмотря на активное развитие, большинство существующих решений (включая архитектуры, аналогичные OpenClaw и другим агентным фреймворкам) остаются ограниченными с точки зрения устойчивости, управляемости и воспроизводимости. Сейчас такие системы демонстрируют нестабильное качество при решении сложных исследовательских задач, что формирует необходимость в разработке архитектурных улучшений, направленных на повышение их эффективности, интерпретируемости и воспроизводимости.

Практическая значимость данной работы заключается в разработке улучшенной архитектуры агентной системы, ориентированной на автоматизацию научного цикла. Предлагаемые изменения позволяют повысить качество работы системы в задачах анализа, генерации и проверки гипотез, что может быть применимо в широком спектре областей - от анализа научных публикаций до поддержки принятия решений в RD. Улучшение механизмов памяти и управления агентами потенциально снижает затраты времени и ресурсов при выполнении сложных интеллектуальных задач. Теоретическая значимость работы связана с вкладом в развитие архитектурных подходов к построению агентных систем. В частности, рассматривается переход от линейных к структурированным представлениям знаний, формализация цикличности научного процесса в агентной логике, а также постановка задачи динамической оптимизации выбора моделей. Эти аспекты расширяют существующие представления о том, как должны быть организованы когнитивные процессы в системах, основанных на языковых

моделях, и могут служить основой для дальнейших исследований в области интеллектуальных агентов.

Целью данной работы является разработка и экспериментальная оценка архитектурных улучшений агентной системы на базе OpenClaw и Clawd-Lab, направленных на повышение ее эффективности при решении исследовательских задач.

Для достижения поставленной цели формулируются следующие задачи:

- проанализировать существующие архитектуры агентных систем и выявить их ограничения, в том числе в контексте решения научных задач.
- разработать механизм структурированной памяти, позволяющий извлекать и хранить информацию в структурном виде для эффективной работы над задачами с большими контекстами
- разработать механизм цикличности, обеспечивающий более гибкое выполнение этапов научного процесса
- реализовать модуль оптимизации выбора наиболее подходящих больших языковых моделей, в зависимости от текущей подзадачи
- интегрировать предложенные компоненты в единую архитектуру
- провести экспериментальную оценку разработанной системы и сравнить ее с базовой архитектурой по заданным критериям

Новизна работы носит прикладной характер и заключается в модификации существующей архитектуры агентной системы с целью устранения выявленных ограничений. В отличие от традиционного подхода, где память агента представлена в виде последовательного лога, предлагается использовать структурированную память, вводится модель цикличности, отражающая природу научного процесса, а также внедряется модуль оптимизации выбора моделей, который динамически адаптирует поведение системы в зависимости от контекста задачи. В совокупности эти изменения формируют более гибкую и адаптивную архитектуру, способную лучше справляться со сложными многошаговыми задачами. Таким образом, работа вносит вклад в область инженерии агентных систем, предлагая конкретные архитектурные решения и демонстрируя их эффективность.

Для оценки эффективности предложенных архитектурных улучшений используется экспериментальный подход, основанный на сравнении базовой и модифицированной систем. Эксперименты проводятся на наборе

задач, имитирующих элементы научного цикла (например, анализ текста, генерация гипотез, выбор методов и интерпретация результатов).

В качестве критериев оценки выделяются три ключевых аспекта:

– качество решений, которое может измеряться через метрики точности, полноты или экспертной оценки результатов
– устойчивость и воспроизводимость, отражающая способность системы получать согласованные результаты при повторных запусках и варьировании входных данных
– эффективность, включая количество шагов, время выполнения и использование ресурсов.

Сравнение проводится в контролируемых условиях с фиксированными сценариями, что позволяет изолировать влияние предложенных изменений. Успехом работы считается значимое улучшение по указанным критериям по сравнению с базовой архитектурой, а также демонстрация более предсказуемого и интерпретируемого поведения системы.

1 АНАЛИЗ ЛИТЕРАТУРЫ

1.1 Структура и организация научных и исследовательских задач

Научные и исследовательские задачи представляют собой класс интеллектуальных задач, направленных на получение новых знаний, проверку гипотез и формирование обоснованных выводов. В отличие от прикладных задач с заранее заданным результатом, научные задачи обычно выполняются в условиях неопределенности: исследователь не всегда заранее знает, какие данные окажутся релевантными, какой метод даст лучший результат и потребуется ли пересмотр исходной гипотезы.

В данной работе научный процесс рассматривается не только как метод познания, но и как вычислительно-организационный процесс, состоящий из связанных этапов: анализа источников, постановки проблемы, формулирования гипотез, планирования проверки, выполнения экспериментов, анализа результатов и подготовки итоговых артефактов. Такая трактовка важна для дальнейшего анализа, поскольку именно многоэтапность и зависимость между этапами делают научные задачи подходящими для автоматизации с использованием мультиагентных систем.

1.1.1 Научный процесс как пайплайн с обратной связью

В общем виде научный процесс можно представить как пайплайн, в котором результат одного этапа становится входом для следующего. Например, анализ литературы формирует представление о текущем состоянии области и нерешенных проблемах, на его основе формулируются гипотезы, далее выбираются методы проверки, проводятся эксперименты и анализируются результаты. При этом такой пайплайн не является строго линейным: результаты эксперимента могут потребовать возврата к этапу постановки гипотезы, изменения метода или повторного анализа источников.

Современные работы по автоматизации научных исследований используют сходное представление. В Agent Laboratory исследовательский процесс разбивается на стадии обзора литературы, экспериментирования и подготовки отчета, а пользователь может давать обратную связь на границах стадий [1]. В AI Scientist-v2 процесс также описывается как

последовательность формулирования гипотез, проектирования и запуска экспериментов, анализа данных и подготовки рукописи, но дополнительно используется агентный поиск по дереву возможных исследовательских траекторий [2]. В pAI/MSc акцент сделан на том, что научная работа должна опираться не только на итоговый текст, но и на проверяемые промежуточные артефакты: гипотезы, планы, результаты вычислений, доказательства и замечания к ним [3].

Таким образом, для автоматизации научного процесса важно учитывать не только последовательность этапов, но и связи между промежуточными результатами. Если система теряет контекст между этапами или не фиксирует причины принятых решений, то последующие шаги становятся менее воспроизводимыми и менее проверяемыми.

1.1.2 Основные этапы и артефакты научной задачи

Для целей данной работы научную задачу целесообразно описывать через набор этапов и артефактов, которые возникают в ходе ее решения. Первый этап связан с анализом предметной области: исследователь изучает публикации, существующие подходы, датасеты, кодовые репозитории и результаты предыдущих экспериментов. Основными артефактами этого этапа являются обзор литературы, список ограничений существующих решений и предварительная формулировка исследовательского разрыва.

Следующий этап включает постановку задачи и формулирование гипотез. На этом этапе из общего анализа выделяется конкретная проблема, задаются критерии успеха и ограничения. В агентных системах этот этап особенно важен, поскольку качество дальнейшего выполнения зависит от того, насколько явно сформулированы цель, ожидаемый результат и проверяемые предположения.

Далее следует планирование и проведение экспериментов. В вычислительных исследованиях оно включает выбор данных, методов, моделей, метрик и последовательности действий. Результатом становятся планы экспериментов, конфигурации запусков, логи выполнения, промежуточные результаты и итоговые метрики. После этого выполняется анализ результатов: проверяется, подтверждают ли они гипотезу, какие ошибки возникли и какие изменения необходимо внести в дальнейшие шаги. Наконец, научный процесс обычно завершается не одним запуском, а

итерацией. Результаты анализа могут привести к уточнению гипотезы, изменению плана или повторной проверке. Поэтому для автоматизации научной задачи недостаточно выполнить фиксированную цепочку действий, необходимо поддерживать сохранение контекста, возврат к предыдущим этапам и проверку согласованности между артефактами.

1.1.3 Проблемы организации научного процесса

Основная сложность организации научных задач связана с тем, что разные этапы требуют разных типов деятельности: поиска информации, рассуждения, планирования, работы с данными, выполнения кода, интерпретации результатов и подготовки текста. В ручном процессе исследователь сам связывает эти этапы между собой, удерживает контекст, принимает решения о возврате к предыдущим шагам и следит за воспроизводимостью.

При использовании программных инструментов эта связность часто нарушается. Данные, заметки, код, результаты экспериментов и выводы могут храниться в разных системах. В результате увеличивается когнитивная нагрузка на исследователя, усложняется проверка решений и возрастает риск потери важного контекста. Для научных задач это особенно критично, поскольку воспроизводимость зависит не только от финального результата, но и от того, насколько прозрачно зафиксированы данные, параметры, промежуточные выводы и причины изменений.

Современные агентные системы пытаются решить эту проблему за счет декомпозиции процесса на роли и этапы. ClawdLab рассматривает научную работу как структурированную лабораторную деятельность с ограничениями ролей, критикой результатов, управлением со стороны principal investigator и требованиями к подтверждению выводов через внешние инструменты [4]. При такой организации остаются следующие вопросы: как сохранять и переиспользовать промежуточный контекст, как обеспечить возврат к предыдущим стадиям и как сделать процесс достаточно гибким без потери управляемости.

1.1.4 Вывод

Научные задачи можно рассматривать как многоэтапные и итеративные процессы, в которых важны не только отдельные действия, но и связи

между ними. Для дальнейшей работы ключевыми являются два свойства такого процесса: необходимость сохранения структурированного состояния исследования и необходимость поддержки обратной связи между этапами.
Именно эти свойства определяют требования к системам автоматизации научных задач. Такая система должна не просто выполнять отдельные операции, а поддерживать исследовательский процесс как связанный набор артефактов, решений и проверок. Это создает основу для дальнейшего анализа существующих инструментов и перехода к мультиагентным архитектурам, в которых отдельные агенты могут отвечать за разные этапы научной работы.
Источники:
1. Agent Laboratory: Using LLM Agents as Research Assistants - https://arxiv.org/abs/2501.04227 2. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search - https://arxiv.org/abs/2504.08066 3. pAI/MSc: ML Theory Research with Humans on the Loop - [https://cbmm.mit.edu/sites/default/files/publications/CBMM4. From Agent-Only Social Networks to Autonomous Scientific Research: Lessons from OpenClaw and Moltbook, and the Architecture of ClawdLab and Beach.Science - https://arxiv.org/abs/2602.19810

1.2 Инструменты автоматизации научных задач

Развитие инструментов для научной деятельности можно рассматривать как переход от автоматизации отдельных операций к попыткам автоматизировать целые исследовательские процессы. Для данной работы важны три группы решений: классические вычислительные инструменты, системы scientific workflows и современные AI/agentic-системы. Они отличаются тем, какую часть научного процесса автоматизируют и насколько хорошо поддерживают связность между этапами.

1.2.1 Инструменты частичной автоматизации

Классические инструменты научных вычислений и анализа данных, такие как Python, R, MATLAB и специализированные библиотеки машинного обучения, позволяют автоматизировать отдельные операции: обработку данных, статистический анализ, моделирование, визуализацию и запуск

вычислительных экспериментов.

Такие инструменты не решают задачу организации научного процесса в целом, они помогают выполнить отдельный шаг, но не определяют, как связать анализ литературы, постановку гипотезы, подготовку данных, эксперимент и интерпретацию результатов. Координация между этапами остается задачей исследователя: он вручную переносит данные между инструментами, фиксирует параметры запусков, оформляет выводы и принимает решения о следующем шаге.

Поэтому классические инструменты являются необходимым, но недостаточным уровнем автоматизации. Они повышают эффективность отдельных операций, но не обеспечивают сквозную воспроизводимость и не поддерживают интеллектуальную координацию исследовательского пайплайна.

1.2.2 Системы автоматизации экспериментов и scientific workflows

Следующим этапом стали системы scientific workflows, предназначенные для описания и выполнения последовательностей научных операций. В таких системах исследователь задает workflow: набор взаимосвязанных шагов, например загрузку данных, предобработку, запуск вычислений и анализ результатов. К этому классу относятся Kepler, Taverna и Galaxy, которые применялись в биоинформатике, вычислительной физике и других областях [1, 2, 3, 4].

Главное преимущество scientific workflows заключается в повышении воспроизводимости. Поскольку шаги процесса, зависимости и параметры фиксируются явно, эксперимент можно повторить, проверить или адаптировать для похожей задачи. Такой подход хорошо работает в случаях, когда структура исследования заранее известна и может быть описана как стабильный пайплайн.

Ограничение данного подхода состоит в его недостаточной гибкости. Научные задачи часто требуют пересмотра гипотез, изменения плана или возврата к предыдущим этапам после получения промежуточных результатов. Классические workflow-системы в основном рассчитаны на заранее заданную последовательность операций и не обладают собственными механизмами рассуждения, критики, выбора следующего шага и интерпретации результатов. Поэтому они решают проблему

воспроизводимого выполнения, но не решают проблему интеллектуальной координации исследования.

1.2.3 Инструменты на базе искусственного интеллекта

Современный этап развития связан с использованием языковых моделей и методов AI for Science. Большие языковые модели показали, что часть когнитивных операций, ранее выполнявшихся человеком, может быть частично автоматизирована: поиск и суммаризация источников, извлечение информации, генерация черновиков текста, объяснение кода и формирование предварительных гипотез [5]. Параллельно развивается направление AI for Science, где методы машинного обучения используются для анализа данных, поиска закономерностей и ускорения научных открытий [6, 7].

Тем не менее, большинство таких инструментов остаются вспомогательными. Они помогают выполнить отдельную задачу, но не образуют единую исследовательскую систему. Например, один инструмент может использоваться для поиска статей, другой - для анализа данных, третий - для генерации текста. Ответственность за согласование результатов, проверку качества и принятие решений между этапами по-прежнему остается на исследователе.

На этом фоне появляются агентные системы, которые пытаются автоматизировать не отдельное действие, а более длинный исследовательский процесс. Agent Laboratory реализует процесс от идеи до отчета через стадии обзора литературы, экспериментов и написания результата [8]. SciAgentGym показывает, что для научных задач важно оценивать способность агента выполнять многошаговое использование инструментов в длинном горизонте [9]. ClawdLab, в свою очередь, рассматривает научную работу как лабораторный процесс с ролями, критикой, управлением и внешней проверкой доказательств [10].

1.2.4 Сравнительный анализ и ограничения существующих подходов

Рассмотренные классы инструментов показывают постепенное развитие автоматизации. Классические вычислительные инструменты эффективны для отдельных операций, но не управляют процессом исследования. Scientific workflow systems позволяют описывать и повторять пайплайны, но требуют заранее заданной структуры и слабо поддерживают пересмотр

решений. Инструменты на базе ИИ автоматизируют когнитивные операции, но часто используются как отдельные сервисы без общего состояния и единой логики выполнения.

Агентные системы являются следующим шагом, поскольку позволяют распределить этапы научного процесса между специализированными компонентами. Для их применения к научным задачам недостаточно просто соединить несколько агентов в последовательность, необходимо обеспечить сохранение артефактов, контроль качества и согласованность между действиями разных агентов.

Таким образом, ключевое ограничение существующих подходов состоит не только в отсутствии автоматизации отдельных операций, а в недостаточной связности всего исследовательского процесса. Именно поэтому в дальнейшем анализе основное внимание уделяется мультиагентным архитектурам и их способности поддерживать структурированную память, цикличность и проверяемость результатов.

1.2.5 Выводы

Существующие инструменты автоматизации научных задач решают разные части общей проблемы. Классические инструменты обеспечивают выполнение вычислений, workflow-системы повышают воспроизводимость фиксированных пайплайнов, а AI-инструменты помогают автоматизировать отдельные когнитивные операции. Но ни один из этих уровней сам по себе не обеспечивает полноценную координацию научного процесса.

Для задач, подобных тем, которые рассматриваются в ClawdLab, наиболее перспективным направлением являются мультиагентные системы. Они позволяют представить исследовательский процесс как взаимодействие специализированных агентов, но требуют дополнительных архитектурных механизмов: общего состояния, структурированной памяти, контроля промежуточных артефактов и возможности итеративного уточнения решений.

Источники:

1. Scientific Workflow Systems - [https://arxiv.org/abs/0907.2580](https://arxiv.org/abs/09
2. The Kepler Scientific Workflow System - https://link.springer.com/article/10.1007/s10723-006-9047-0 3. Taverna:

a tool for the composition and enactment of bioinformatics workflows - [https://academic.oup.com/bioinformatics/article/22/24/3167/206046](https://academic.o
4. Galaxy platform - https://galaxyproject.org/
5. GPT-3: Language Models are Few-Shot Learners - https://arxiv.org/abs/2005.14165 6. Artificial Intelligence for Science - https://www.nature.com/articles/s41586-023-06221-2 7. Machine Learning for Scientific Discovery - https://arxiv.org/abs/1903.10563 8. Agent Laboratory: Using LLM Agents as Research Assistants - https://arxiv.org/abs/2501.04227 9. SciAgentGym: Benchmarking Multi-Step Scientific Tool-use in LLM Agents - https://arxiv.org/abs/2602.12984 10. From Agent-Only Social Networks to Autonomous Scientific Research: Lessons from OpenClaw and Moltbook, and the Architecture of ClawdLab and Beach.Science - https://arxiv.org/abs/2602.19810

1.3 Автоматизация научных задач агентными системами

Развитие больших языковых моделей привело к появлению нового класса программных систем - интеллектуальных агентов, способных не только генерировать текст, но и планировать действия, вызывать внешние инструменты, анализировать промежуточные результаты и корректировать дальнейшее поведение. В контексте научных задач это особенно важно, поскольку исследовательский процесс состоит из взаимосвязанных этапов: анализа литературы, формулирования гипотез, планирования экспериментов, выполнения вычислений, анализа результатов и подготовки итоговых материалов.

Под агентом в данной работе понимается программный компонент, который получает цель или задачу, использует языковую модель для рассуждения и может выполнять действия во внешней среде: обращаться к инструментам, запускать код, работать с данными, сохранять результаты и взаимодействовать с другими агентами. Мультиагентная система, в свою очередь, состоит из нескольких таких компонентов, между которыми распределены роли, задачи и ответственность за отдельные части процесса.

1.3.1 От одиночных агентов к мультиагентным системам

Первые работы по агентам на базе языковых моделей были в основном направлены на объединение рассуждения и действий. В подходе ReAct предложено чередовать шаги рассуждения и использования инструментов, что позволяет агенту не только формировать ответ, но и проверять промежуточные предположения через внешнюю среду [1]. В Toolformer показано, что языковая модель может обучаться использовать внешние инструменты, например поисковые системы, калькуляторы или API, для повышения точности выполнения задач [2]. Эти подходы стали основой для дальнейшего развития агентных систем, поскольку они рассматривают модель как управляющий компонент в более широкой вычислительной среде.

Другим важным направлением стало добавление памяти и долгосрочного контекста. В Generative Agents память используется для хранения событий и последующего извлечения релевантной информации, что позволяет агентам демонстрировать более последовательное поведение на длинном горизонте взаимодействия [3]. Для научных задач это свойство является принципиальным: исследовательская работа редко состоит из одного запроса, а промежуточные выводы, параметры экспериментов и замечания к результатам должны сохраняться и использоваться на следующих этапах.

Ранние автономные системы, такие как AutoGPT и BabyAGI, продемонстрировали возможность построения цепочек действий без постоянного участия пользователя. Но их применение выявило ряд ограничений: слабую устойчивость к ошибкам, накопление неточностей в длинных цепочках рассуждений, отсутствие строгого контроля качества и недостаточную воспроизводимость. Для научных задач эти ограничения особенно критичны, поскольку ошибочный промежуточный вывод может повлиять на последующие эксперименты и привести к некорректной интерпретации результатов.

Переход к мультиагентным системам связан с попыткой уменьшить эти ограничения за счет разделения процесса на специализированные роли. Вместо одного универсального агента система может включать агента-исследователя, агента-инженера, агента-критика, агента-аналитика и другие компоненты. Такое разделение повышает модульность и позволяет формализовать отдельные этапы работы. При этом возникает новая

задача: необходимо не только создать набор агентов, но и определить, как они координируются, где хранится общее состояние, как проверяются результаты и каким образом система возвращается к предыдущим этапам при обнаружении ошибок.

1.3.2 Современные архитектуры LLM-агентов и их ограничения

Современные обзоры LLM-агентов показывают, что развитие таких систем связано не только с ростом качества базовых языковых моделей, но и с усложнением внешней архитектуры вокруг модели. Типовая агентная система включает несколько ключевых компонентов: модуль планирования, память, механизм вызова инструментов, протокол взаимодействия с внешней средой и блок оценки промежуточных результатов [4]. В мультиагентных системах к этим компонентам добавляются коммуникация между агентами, распределение ролей и механизмы координации.

Одним из центральных архитектурных компонентов является память. В ранних агентных системах память часто представлялась как история сообщений или набор текстовых артефактов. Для сложных задач этого недостаточно: агенту необходимо не только хранить факты, но и понимать, какие выводы были сделаны ранее, какие данные их поддерживают, какие утверждения были опровергнуты и какие решения требуют пересмотра. В современных обзорах память рассматривается как отдельный цикл записи, управления и чтения (write-manage-read), который влияет на устойчивость агента при длительном выполнении задач [7].

Более сильный вариант этой идеи представлен в подходах к графовой когнитивной памяти. В работе Graph-Native Cognitive Memory память агента моделируется как граф, содержащий утверждения, версии, зависимости, отношения поддержки и противоречия [8]. Такой подход позволяет представить память как часть рассуждения агента. Для научных задач это особенно важно, поскольку результаты экспериментов, гипотезы, аргументы и замечания критиков должны быть связаны между собой машинно-обрабатываемым образом.

Другой важный компонент современных агентных систем - коммуникация между агентами. В мультиагентных системах качество результата зависит не только от способности отдельного агента рассуждать, но и от того, как агенты обмениваются информацией, распределяют ответственность

и разрешают противоречия. Обзоры коммуникационно-ориентированных мультиагентных систем показывают, что при росте числа агентов появляются проблемы координации, избыточного обмена сообщениями, потери фокуса и накопления ошибок в коллективном рассуждении [5].

Отдельной проблемой является оценка качества агентных систем. В отличие от обычных задач генерации текста, здесь важно оценивать не только финальный ответ, но и весь процесс выполнения: корректность плана, использование инструментов, устойчивость памяти, способность к самопроверке и обработке ошибок [6]. Это особенно существенно для длинных цепочек действий, где одна ошибка на раннем шаге может повлиять на все последующие решения.

Таким образом, основные ограничения современных агентных систем связаны с потерей контекста, ошибками в длинных цепочках рассуждений, недостаточной воспроизводимостью, сложностью координации и слабой формализацией промежуточного состояния. Эти проблемы напрямую связаны с задачами, рассматриваемыми в данной работе. Структурированная память позволяет уменьшить потерю контекста и сделать промежуточные выводы проверяемыми, а циклическое выполнение задач позволяет возвращаться к предыдущим шагам при обнаружении ошибки или недостаточного качества результата.

1.3.3 Агентные системы для научного процесса

В последние годы появились работы, напрямую ориентированные на автоматизацию научной деятельности. Их можно разделить на несколько групп: системы полного исследовательского цикла, системы для многошагового использования инструментов, системы подготовки научных публикаций и лабораторные платформы с управлением ролями и проверкой результатов.

Одним из наиболее известных направлений являются системы полного исследовательского цикла. В Agent Laboratory исследовательский процесс организован вокруг трех основных стадий: обзор литературы, экспериментирование и подготовка отчета [9]. В рамках каждой стадии используются агенты с разными ролями, а пользователь может давать обратную связь между стадиями. Достоинство такого подхода состоит в том, что он явно приближает структуру системы к реальной исследовательской

практике. Ограничение заключается в том, что процесс во многом остается стадийным: переходы между этапами заданы заранее, а возврат к предыдущим решениям требует отдельной организации.

Система AI Scientist-v2 развивает идею автоматизированного научного открытия через генерацию идей, проектирование экспериментов, запуск кода, анализ результатов и подготовку статьи [10]. Важной особенностью является агентный поиск по дереву (agentic tree search), который позволяет исследовать несколько исследовательских траекторий, а не следовать единственному заранее выбранному пути. Это делает систему более близкой к реальному научному процессу, где гипотезы конкурируют между собой и могут уточняться по результатам проверки.

Отдельный интерес представляет DeepScientist, ориентированный на целевое автономное научное открытие в течение длительного времени [11]. В этой работе процесс открытия формализуется как задача байесовской оптимизации (Bayesian Optimization), а сама система работает в цикле ”гипотеза - проверка - анализ” (hypothesize - verify - analyze). Важным архитектурным элементом является накопительная память находок (Findings Memory), в которой сохраняются результаты предыдущих проверок. За счет этого система балансирует исследование новых гипотез и углубление перспективных направлений. Для данной работы DeepScientist важен как пример того, что в современных системах научной автоматизации критически значимыми становятся два механизма: накопление структурированного состояния и циклическое уточнение гипотез.

Другая группа работ фокусируется на многошаговом использовании инструментов. SciAgentGym рассматривает научную задачу как длинный горизонт взаимодействия, где агент должен последовательно выбирать действия, использовать инструменты и интерпретировать результаты [12]. Эта работа показывает, что качество агентных систем нельзя оценивать только по финальному ответу: важно учитывать устойчивость многошагового процесса, корректность вызова инструментов и способность системы не терять контекст между операциями.

В pAI/MSc научный процесс рассматривается как совместная работа человека и системы, где человек остается ”в контуре” или ”над контуром”

управления (human-on-the-loop), а агентная система помогает формировать гипотезы, проверять утверждения, фиксировать промежуточные артефакты и готовить научный текст [13]. Такой подход особенно важен для задач, где полностью автономное открытие пока недостаточно надежно, но автоматизация отдельных этапов уже может повысить скорость и качество работы.

Система aiXiv ориентирована преимущественно на подготовку и улучшение научных публикаций [14]. В ней основным артефактом является документ, который постепенно модифицируется агентами. Этот подход можно рассматривать как частный случай общей памяти (shared memory), где центральным объектом является не все состояние исследования, а текстовая рукопись. Для данной работы aiXiv полезен как пример того, что научные агентные системы могут быть организованы вокруг артефактов, а не только вокруг сообщений между агентами.

Таким образом, related work показывает общий сдвиг от одиночных агентов к системам, которые моделируют исследовательский процесс как последовательность ролей, артефактов, проверок и итераций.

1.3.4 Архитектурные подходы к построению научных агентных систем

С точки зрения архитектуры можно выделить несколько типовых подходов к построению мультиагентных систем для научных задач.

Первый подход - ролевые архитектуры (role-based architectures). В таких системах агенты имеют заранее определенные функции: например, исследователь, инженер, аналитик, рецензент или руководитель эксперимента. Этот подход используется в Agent Laboratory и близких системах, где роли соответствуют этапам научной работы [9]. Его преимущество состоит в понятности и управляемости: каждую роль можно описать через инструкции, входы и ожидаемые выходы. Недостаток заключается в ограниченной гибкости: если задача требует нестандартного маршрута, фиксированная последовательность ролей может мешать возврату к предыдущим этапам или изменению плана.

Второй подход - архитектуры с оркестратором (orchestrator-based architectures). В них центральный компонент отвечает за декомпозицию задачи, назначение агентов, контроль выполнения и хранение глобального

состояния [15]. Такой подход хорошо подходит для научных задач, где важно фиксировать зависимости между действиями и воспроизводить ход эксперимента. Оркестратор может хранить список задач, промежуточные результаты, логи и метаданные. Ограничение состоит в том, что центральный компонент становится сложным и должен явно управлять состоянием, ошибками и возвратом к предыдущим этапам.

Третий подход - графовые архитектуры взаимодействия (graph-based architectures). В таких системах агенты представлены как узлы графа, а связи между ними - как каналы передачи данных или сообщений [16]. Графовая организация позволяет описывать сложные зависимости между задачами и запускать некоторые операции параллельно. При этом состояние часто распределено между узлами, что повышает гибкость, но усложняет контроль целостности данных и воспроизводимость.

Четвертый подход - адаптивные пайплайны (adaptive pipelines). В таких системах структура выполнения может изменяться в зависимости от промежуточных результатов [17]. Например, компонент оценки может принять решение о повторе эксперимента, добавлении нового шага или возврате к формулировке гипотезы. Этот подход особенно близок к научному процессу, поскольку исследовательская работа редко следует единственному линейному маршруту.

Пятый подход - динамическая генерация архитектуры (dynamic architecture generation). В нем мета-агент формирует набор ролей, структуру взаимодействия и порядок выполнения под конкретную задачу [18]. Такой подход обеспечивает максимальную гибкость, но снижает предсказуемость и усложняет проверку результатов. Для научных задач это может быть перспективным направлением, но в рамках данной работы более важны контролируемые и воспроизводимые доработки существующей системы.

1.3.5 Сравнение архитектурных подходов

Для анализа применимости перечисленных подходов к научным задачам удобно сравнивать их не только по общей гибкости, но и по тому, как они поддерживают состояние исследования, обратную связь и проверку результатов. Текстовое сравнение позволяет объяснить компромиссы, а таблица фиксирует основные различия в компактном виде.

С точки зрения данной работы наиболее важны не все возможные

Таблица 1.1 — Сравнение архитектурных подходов к построению научных агентных систем

Подход	**Управление и координация**	**Память и состояние**	**Поддержка итераций**	**Ограничения**
Ролевая архитектура	Последовательность специализированных ролей, управление часто задается заранее	Общий контекст или набор артефактов между стадиями	Возможна через обратную связь пользователя или отдельного агента-критика	Ограниченная гибкость при нестандартных исследовательских траекториях
Архитектура с оркестратором	Центральный компонент назначает задачи, контролирует зависимости и выполнение	Централизованное состояние, логи, список задач и результатов	Может быть реализована явно через возврат задач в очередь или повтор этапов	Высокая сложность оркестратора и необходимость строгого управления состоянием
Графовая архитектура	Выполнение определяется структурой графа и потоками сообщений	Распределенное состояние узлов и передаваемых артефактов	Возможны циклы в графе и параллельные ветви рассуждения	Сложнее обеспечить глобальный контроль, отладку и воспроизводимост
Адаптивный пайплайн	Компонент оценки изменяет последовательность шагов по результатам выполнения	Состояние пайплайна, история изменений, результаты оценки	Поддерживается естественно через повтор, пересмотр и добавление шагов	Требует формализации критериев изменения пайплайна и хорошего логирования
Динамическая генерация архитектуры	Мета-агент создает конфигурацию системы под задачу	Мета-память о шаблонах, конфигурациях и результатах	Зависит от сгенерированной структуры	Высокая гибкость при низкой предсказуемости и сложности валидации

архитектурные свойства, а те, которые связаны с ограничениями ClawdLab. Система должна сохранять промежуточные результаты не только как набор файлов или сообщений, но как структурированное состояние исследования. Она должна поддерживать возврат к предыдущим этапам, если критика или результаты проверки показывают, что исходная гипотеза или план требуют уточнения. Такая цикличность не должна разрушать управляемость: каждый возврат и каждое изменение должны фиксироваться, чтобы процесс оставался воспроизводимым.

1.3.6 Выводы

Обзор related work показывает, что современные агентные системы для научных задач постепенно переходят от линейных пайплайнов к более сложным архитектурам, включающим роли, общую память, критику, внешнюю проверку и циклическое уточнение решений. Наиболее релевантными для данной работы являются системы, в которых научный процесс представлен как набор артефактов и итераций, а не только как последовательность сообщений между агентами.

При этом ни один из рассмотренных подходов не решает все проблемы одновременно. Ролевые системы понятны и управляемы, но часто недостаточно гибки. Оркестраторные системы дают лучший контроль, но требуют сложного управления состоянием. Графовые и адаптивные подходы повышают гибкость, но усложняют воспроизводимость. Поэтому дальнейшее рассмотрение OpenClaw и ClawdLab необходимо для того, чтобы определить, какие архитектурные идеи можно перенести в конкретную систему и какие ограничения следует устранить в первую очередь.

Источники:

1. ReAct: Synergizing Reasoning and Acting in Language Models - https://arxiv.org/abs/2210.03629 2. Toolformer: Language Models Can Teach Themselves to Use Tools - https://arxiv.org/abs/2302.04761 3. Generative Agents: Interactive Simulacra of Human Behavior - https://arxiv.org/abs/2304.03442 4. LLMs Working in Harmony: A Survey on the Technological Aspects of Building Effective LLM-Based Multi Agent Systems - https://arxiv.org/abs/2504.01963 5. Beyond Self-Talk: A Communication-Centric Survey of LLM-Based Multi-Agent Systems - https://arxiv.org/abs/2502.14321 6. Survey on Evaluation of LLM-based Agents - https://arxiv.org/abs/2503.16416 7. Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers - https://arxiv.org/abs/2603.07670 8. Graph-Native Cognitive Memory for AI Agents - https://arxiv.org/abs/2603.17244 9.

Agent Laboratory: Using LLM Agents as Research Assistants - https://arxiv.org/abs/2501.04227 10. The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search - https://arxiv.org/abs/2504.08066 11. DeepScientist: Advancing Frontier-Pushing Scientific Findings Progressively - https://arxiv.org/abs/2509.26603 12. SciAgentGym: Benchmarking Multi-Step Scientific Tool-use in LLM Agents - https://arxiv.org/abs/2602.12984 13. pAI/MSc: ML Theory Research with Humans on the Loop - [https://cbmm.mit.edu/sites/default/files/publications/CBMM14. aiXiv - https://arxiv.org/abs/2508.15126 15. Orchestrator-based Agents - [https://arxiv.org/abs/2602.03786](https://arxiv.org/abs/2602.
16. Graph-based Agent Interaction - [https://arxiv.org/html/2603.22386v1](https://arxiv.or
17. Adaptive Pipelines - [https://arxiv.org/pdf/2601.12542](https://arxiv.org/pdf/2601.125
18. Dynamic Architecture Generation - [https://arxiv.org/pdf/2602.01550](https://arxiv.org

1.4 Фреймворк OpenClaw и система ClawdLab

После обзора общих подходов к построению научных агентных систем необходимо отдельно рассмотреть OpenClaw и ClawdLab, поскольку именно они являются исходной технологической основой для дальнейшего совершенствования. В отличие от универсальных работ из related work, эти системы важны не только как примеры архитектурных подходов, но и как конкретный объект анализа AS IS.

1.4.1 OpenClaw как инфраструктурная основа

OpenClaw представляет собой открытый самохостинговый шлюз (self-hosted gateway) для подключения AI-агентов к различным каналам взаимодействия: мессенджерам, веб-интерфейсам, мобильным узлам и внешним плагинам [1]. С точки зрения данной работы OpenClaw важен как инфраструктурный слой, который предоставляет механизмы подключения агентов, управления сессиями, маршрутизации, работы с инструментами и организации взаимодействия через разные каналы.

В документации OpenClaw подчеркивается несколько ключевых свойств: мультиканальность, поддержка плагинов, агентно-ориентированная архитектура, изолированные сессии и маршрутизация между агентами

[1]. Это делает OpenClaw удобной основой для построения более специализированных систем. Например, поверх него можно реализовывать разные сценарии взаимодействия: персонального ассистента, командного агента, систему с несколькими агентами или лабораторную среду.

Архитектурно OpenClaw можно рассматривать как слой, который отделяет пользовательские каналы и интерфейсы от агентной логики. Пользователь или внешний сервис отправляет сообщение через канал, шлюз сопоставляет его с нужной сессией и агентом, после чего агент выполняет задачу с использованием доступных инструментов и памяти. Такая организация позволяет подключать агентов к разным интерфейсам без необходимости каждый раз заново реализовывать коммуникационный слой.

Для исследовательских задач особенно важны следующие компоненты OpenClaw:

- маршрутизация между агентами и сессиями, позволяющая разделять контексты разных задач, - поддержка инструментов и плагинов, через которые агенты могут получать доступ к внешним функциям, - возможность работы с медиа и документами, что важно для анализа научных материалов, - управление сессиями и состоянием взаимодействия, - возможность развертывания на собственной инфраструктуре, что повышает контроль над данными.

Таким образом, OpenClaw предоставляет полезную инженерную основу, но сам по себе не задает полный научный процесс. Он не определяет, какие роли должны быть в исследовательской системе, как проверять гипотезы, как хранить промежуточные научные артефакты и когда возвращаться к предыдущим этапам. Эти вопросы решаются на уровне специализированных надстроек, одной из которых является ClawdLab.

1.4.2 ClawdLab как лабораторная надстройка над OpenClaw

ClawdLab описывается как платформа для структурированной лабораторной кооперации агентов, созданная на основе опыта OpenClaw и Moltbook [2]. В статье ClawdLab рассматривается как ответ на архитектурные проблемы, возникающие в системах, где агенты взаимодействуют свободно и без достаточного контроля. К таким проблемам относятся отсутствие строгих ролей, зависимость от социального согласия агентов, слабая проверяемость выводов и недостаточная управляемость исследовательского процесса.

В качестве базовой единицы научной деятельности в ClawdLab используется задача (task). В статье выделяются пять типов таких задач: обзор литературы (literature review), анализ (analysis), глубокое исследование (deep research), критика (critique) и синтез (synthesis) [2]. Обзор литературы связан с поиском источников, выделением известных методов, результатов и ограничений. Аналитические задачи включают обработку данных, статистическое моделирование, построение графиков, проверку воспроизводимости и оформление вычислительных артефактов. Глубокое исследование предполагает более развернутую проработку исследовательского вопроса, где требуется не только собрать факты, но и связать их с гипотезой, данными и возможными способами проверки. Критика используется для выявления ошибок, слабых мест и альтернативных интерпретаций уже выполненной работы. Синтез объединяет результаты обзора, анализа и критики в связный исследовательский артефакт, например отчет или документ лаборатории.

Эти типы задач связаны с ролевой моделью ClawdLab. Например, агент-разведчик (scout) выполняет задачи обзора литературы, исследователь-аналитик (research analyst) отвечает за анализ и глубокое исследование, критик (critic) выполняет задачи критики, а синтезатор (synthesizer) объединяет результаты в итоговые документы. Главный исследователь (principal investigator) управляет постановкой и принятием задач, может инициировать голосование и завершать исследовательские состояния [2]. Такая организация важна для данной работы, поскольку показывает, что ClawdLab уже содержит предметную декомпозицию научного процесса, а предлагаемые далее улучшения должны усиливать выполнение этих типов задач через память и цикличность, а не заменять исходную ролевую архитектуру.

В отличие от полностью свободных агентных сред, ClawdLab вводит более строгую организацию научной работы. В системе используются жесткие ограничения ролей (hard role restrictions), структурированная состязательная критика (structured adversarial critique), управление со стороны главного исследователя (principal investigator-led governance) и требования к доказательствам, проверяемым через внешние инструменты [2]. Это означает, что результат работы агента не должен приниматься только потому, что другие агенты с ним согласились, он должен быть подтвержден

доступными API, вычислительными сервисами, инструментами или другим внешним механизмом проверки.

Согласно статье, ClawdLab связан с более широкой таксономией агентных научных систем. В ней выделяются одиночные агентные пайплайны, заранее определенные мультиагентные рабочие процессы и более децентрализованные системы [2]. ClawdLab занимает промежуточную позицию: он стремится сохранить управляемость лабораторного процесса, но при этом использует несколько агентов, разные роли, инструменты и механизмы проверки. Для данной работы это важно, поскольку совершенствование ClawdLab должно сохранять его сильные стороны - контроль, проверяемость и управляемость, - но при этом уменьшать ограничения линейного выполнения и пассивного хранения артефактов.

1.4.3 Текущая организация процесса в ClawdLab

ClawdLab можно рассматривать как систему, ориентированную на поддержку научной работы от генерации идей до их последующей проработки. В обобщенном виде процесс включает два крупных этапа.

Первый этап связан с генерацией и обсуждением исследовательских идей. На этом этапе агенты и человек могут формировать потенциальные направления, обсуждать их ценность и выбирать наиболее перспективные. Такой подход позволяет использовать языковые модели для расширения пространства идей, но при этом сохранять экспертный контроль.

Второй этап связан с более структурированной проработкой выбранной идеи. Здесь используются агенты с фиксированными ролями: планирование, выполнение, критика, проверка и подготовка результата. Взаимодействие чаще всего организуется как управляемый процесс, где результаты одного этапа передаются дальше и становятся входом для следующего. Это повышает предсказуемость, но может ограничивать гибкость, если после проверки требуется вернуться к ранним предположениям.

В ClawdLab присутствует общее пространство артефактов: результаты выполнения задач, обсуждения агентов, журналы активности и документы проекта. Такой слой важен для координации, поскольку он позволяет участникам системы обращаться к уже созданным материалам. Но в текущем виде такое хранилище можно рассматривать скорее как репозиторий артефактов, чем как активную структурированную память, встроенную в

процесс принятия решений. Иными словами, информация сохраняется, но не всегда явно включается в рассуждение агентов и не всегда имеет формализованные связи с гипотезами, задачами, проверками и выводами.

1.4.4 Сильные стороны и ограничения OpenClaw/ClawdLab

К сильным сторонам OpenClaw относится инженерная расширяемость. Система предоставляет шлюз, каналы, сессии, маршрутизацию, плагины и инструменты. Это позволяет использовать ее как основу для построения прикладных агентных систем. В контексте научных задач важно, что OpenClaw не жестко привязан к одному интерфейсу взаимодействия и допускает подключение разных агентов и внешних инструментов.

Еще к сильным сторонам ClawdLab относится более предметная организация научного процесса. В системе появляются роли, управление, критика, голосование, журналы активности, provider jobs и артефакты, которые создают основу для последующей проверки результатов. Это делает ее ближе к реальной исследовательской лаборатории, чем простые агентные пайплайны. ClawdLab также явно учитывает проблему доверия к выводам агентов: результат должен быть не только сгенерирован, но и связан с проверяемыми материалами.

При этом остаются ограничения, важные для данной работы. Первое ограничение связано с организацией памяти. Общее хранилище артефактов помогает сохранять результаты, но не обязательно превращает их в структурированное состояние исследования. Для научного процесса важно видеть связи между гипотезами, аргументами, экспериментами, ошибками и итоговыми выводами. Без такой структуры агент может использовать сохраненную информацию непоследовательно или терять важный контекст.

Также в системе цикличность выполнения подзадач неформализована и не стандартизирована. Научная работа требует возврата к предыдущим этапам: гипотеза может быть отклонена, план эксперимента может потребовать пересмотра, а результаты анализа могут выявить ошибку в исходной постановке. Если процесс в системе организован преимущественно как последовательный пайплайн, то такие возвраты становятся неявными и требуют ручной координации.

Отдельное ограничение связано с расхождением между архитектурными заявлениями в статье и текущей реализацией main-ветки ClawdLab. В

статье утверждается, что результаты в ClawdLab валидируются через domain-specific evidence requirements и external tool verification, а качество научного вывода определяется результатами вычислительных инструментов, а не социальным консенсусом [2]. Но анализ текущего репозитория показывает, что в main-ветке это реализовано скорее как инфраструктурная заготовка: есть provider jobs для literature и analysis, task votes, critiques, activity log и поле verificationScore, но нет полноценного механизма доменно-специфической автоматической проверки результата, статусов verification, verificationResult, правил принятия результата по evidence requirements и формализованного связывания вывода с проверочным артефактом [3]. Существовала отдельная доработка/PR, где появлялись поля и API для verification, но она не была принята в main-ветку, следовательно, такую проверку нельзя считать реализованной возможностью текущей версии системы.

Похожий overclaim наблюдается и в других утверждениях статьи. Там multi-model orchestration описывается как свойство ClawdLab, но на уровне платформы сейчас фиксируется только метаданное foundationModel у агента, а выбор модели и ее замена остаются во внешнем runtime агента [3]. Поэтому ClawdLab может быть совместим с агентами на разных моделях, но не управляет модельной маршрутизацией сам. Еще emergent Sybil resistance в статье представлена как структурное следствие ролей, PI-led governance и evidence requirements, но сами авторы признают, что система на момент публикации работала с агентами команды разработчиков и не была проверена на независимых внешних агентах, adversarial inputs и долгосрочной эксплуатации [2]. Поэтому эту устойчивость корректнее рассматривать как архитектурное предположение, а не подтвержденное свойство реализации.

Еще одно практическое ограничение касается интеграции внешних вычислительных сервисов. В README репозитория заявлено, что backend owns secrets и агенты не должны получать provider API keys или storage credentials [3]. Но в файле ANALYSIS$_{L}IMITATIONS.mdS3bucketmismatch:$ $S3credentialsrequestbody,,[4].,,backend-.$

Эти ограничения согласуются с выводами современной литературы по научным агентным системам. SciAgentGym показывает, что при увеличении горизонта взаимодействия качество tool-use в научных задачах заметно

падает, поэтому простого наличия внешних инструментов недостаточно: важны устойчивое управление процессом, трассируемость действий и проверяемость промежуточных шагов [5]. Работы по ultra-long-horizon agentic science дополнительно показывают, что агентам нужна не только история сообщений, а многоуровневая память, позволяющая отделять краткосрочные следы выполнения от устойчивого исследовательского знания [6]. Следовательно, реальные ограничения ClawdLab состоят не только в отсутствии отдельных API, но и в том, что текущая архитектура пока не обеспечивает полноценную связку ”гипотеза - попытка - внешний результат - критика - решение - пересмотр”, необходимую для воспроизводимого научного процесса.

1.4.5 Связь с дальнейшим образом решения TO BE

Анализ OpenClaw и ClawdLab показывает, что выбранная технологическая основа уже содержит важные элементы для построения научной мультиагентной системы: агенты, инструменты, сессии, роли, управление, артефакты и механизмы проверки. Поэтому задача данной работы состоит не в создании системы с нуля, а в развитии существующей архитектуры в направлении, которое следует из related work.

Первое направление развития - структурированная память. Она должна связывать артефакты исследования между собой: гипотезы, задачи, результаты экспериментов, замечания критиков, решения о пересмотре и итоговые выводы. Такая память должна быть не только хранилищем, но и частью процесса рассуждения, чтобы агенты могли явно опираться на предыдущие результаты.

Второе направление развития - циклическая модель исследовательского процесса. Она должна позволить системе возвращаться к предыдущим этапам при появлении новой информации или критики. При этом каждый возврат должен фиксироваться как часть истории исследования, чтобы сохранить воспроизводимость и возможность анализа ошибок.

Таким образом, дальнейшая гипотеза работы может быть сформулирована вокруг того, что расширение ClawdLab структурированной памятью и циклическим процессом позволит повысить согласованность, воспроизводимость и качество решений в задачах научного анализа и проработки гипотез.

Источники:

1. OpenClaw documentation - https://docs.openclaw.ai/
2. From Agent-Only Social Networks to Autonomous Scientific Research: Lessons from OpenClaw and Moltbook, and the Architecture of ClawdLab and Beach.Science - https://arxiv.org/abs/2602.19810
3. ClawdLab GitHub repository - https://github.com/bio-xyz/ClawdLab 4. ClawdLab ANALYSIS$_L IMITATIONS.md - https://github.com/bio-xyz/ClawdLab/blob/main/ANALYSIS_L IMITATIONS.md5. Benchmarking Multi-Step Scientific Tool-use in LLM Agents - https://arxiv.org/abs/2602.129846. Toward Ultra-Long-Horizon Agentic Science: Cognitive Accumulation for Machine Learning Engineering https://arxiv.org/abs/2601.10402

1.5 Образ целевого решения TO BE и архитектурная гипотеза

Проведенный анализ показывает, что OpenClaw и ClawdLab уже содержат важные элементы для построения научной мультиагентной системы: агентов, роли, инструменты, сессии, артефакты, обсуждения, журналы активности и механизмы проверки. Поэтому в данной работе не ставится задача создания новой агентной платформы с нуля. Цель состоит в развитии существующей архитектуры ClawdLab в направлении, которое следует из анализа научного процесса и related work: усиление памяти исследования и добавление управляемой цикличности выполнения задач.

1.5.1 Переход от AS IS к TO BE

В текущем состоянии ClawdLab можно рассматривать как систему, в которой научная работа организована вокруг задач, ролей и артефактов. Такой подход обеспечивает управляемость и позволяет фиксировать результаты выполнения. Но связь между отдельными артефактами остается преимущественно неявной: результаты задач, обсуждения, критика и документы сохраняются, но не всегда образуют единое структурированное состояние исследования.

Кроме того, процесс выполнения задачи в базовой конфигурации близок к последовательному пайплайну. Система может поддерживать обсуждения, критику и создание новых задач, но возврат к предыдущим этапам внутри одной исследовательской задачи не представлен как отдельный формализованный механизм. В результате итеративная природа научного процесса частично переносится на пользователя или внешнюю организацию работы.

Целевое состояние TO BE предполагает сохранение сильных сторон ClawdLab: ролевой организации, управляемости, внешней проверки и работы с артефактами. При этом система должна быть дополнена двумя архитектурными свойствами:

- структурированной памятью исследования, - циклической моделью выполнения задач.

Эти свойства не заменяют существующую архитектуру, а расширяют ее. Структурированная память должна сделать связи между исследовательскими артефактами явными, а циклическая модель должна позволить системе возвращаться к предыдущим шагам без потери воспроизводимости и управляемости.

1.5.2 Структурированная память исследования

Под структурированной памятью исследования в данной работе понимается не просто хранилище файлов, сообщений или логов, а организованное представление состояния научной задачи. Такая память должна фиксировать основные сущности исследовательского процесса и связи между ними: гипотезы, задачи, попытки выполнения, входные данные, результаты экспериментов, замечания критиков, решения о пересмотре и итоговые выводы.

В базовом варианте ClawdLab уже хранит значительную часть этих данных в виде артефактов, обсуждений, журналов активности и документов. Но для решения важно, чтобы эти элементы были связаны между собой и могли использоваться агентами в дальнейшем рассуждении. Например, итоговый вывод должен быть связан с гипотезой, которую он проверяет, с попыткой выполнения, в рамках которой он был получен, с критикой, которая выявила ограничения, и с решением о принятии или повторной проверке результата.

Такая память выполняет несколько функций. Она уменьшает потерю

контекста между этапами исследованиях, повышает проверяемость выводов и создает основу для циклического процесса: если новая попытка выполнения создается после критики, система должна понимать, какая именно проблема исправляется и какие предыдущие результаты необходимо учесть.

На концептуальном уровне структурированная память может быть представлена как слой, связывающий существующие объекты Clawd-Lab. Конкретный способ реализации этого слоя, например через отдельные таблицы, графовые связи или иерархическую модель памяти, относится к следующей главе, где рассматриваются технологии и экспериментальный дизайн.

1.5.3 Циклическая модель выполнения задач

Вторым направлением развития является переход от преимущественно линейного выполнения к управляемой циклической модели. В научной работе результат первого прохода редко является окончательным: гипотеза может оказаться слишком общей, экспериментальный план может потребовать уточнения, а анализ результатов может выявить ошибку в исходных данных или методике. Поэтому целевая система должна поддерживать не только завершение задачи, но и ее повторное уточнение.

Для описания такой модели целесообразно разделить исследовательскую задачу и попытку ее выполнения. Задача (Task) задает исследовательскую цель: например, проанализировать подход, проверить гипотезу или подготовить вывод по набору источников. Попытка выполнения (Attempt или Run) фиксирует один конкретный проход решения этой задачи: использованный план, входные данные, действия агентов, полученные артефакты, замечания критиков и итоговое решение.

Если результат попытки признается недостаточным, система не должна терять предыдущий контекст или неформально начинать задачу заново. Вместо этого создается новая попытка, связанная с предыдущей. В ней явно фиксируется, что именно изменилось: уточнена гипотеза, заменены данные, добавлен новый источник, изменен план проверки или учтена критика. Таким образом, научный процесс становится не хаотическим набором повторных запусков, а последовательностью документированных итераций.

Циклическая модель может включать следующие концептуальные шаги:
- постановка исследовательской задачи, - формирование первой попытки выполнения, - получение промежуточных и итоговых артефактов, - критика и проверка результата, - решение о принятии результата, доработке или повторной проверке, - создание следующей попытки при необходимости.
При этом цикличность должна оставаться управляемой. Для научной системы важно не только разрешить возврат к предыдущим этапам, но и фиксировать причины такого возврата, условия остановки и изменения относительно предыдущих попыток. Это позволяет сохранить воспроизводимость и сделать процесс анализа ошибок более прозрачным.

1.5.4 Границы целевого решения

В related work также рассматриваются другие направления развития агентных систем, включая многомодельную оркестрацию (multi-model orchestration), адаптивный выбор моделей и динамическую генерацию архитектуры. Эти направления являются перспективными, но в рамках данной работы они не включаются в экспериментальную доработку.
Причина этого ограничения связана с архитектурой ClawdLab. Система предполагает подключение внешних агентов, каждый из которых может использовать собственную модель и собственный исполнительный контур. ClawdLab регистрирует агента, задает его роль и обеспечивает взаимодействие с ним, но не управляет напрямую внутренним выбором модели при выполнении конкретной подзадачи. Поэтому механизм динамического выбора моделей рассматривается как возможное направление дальнейшего развития, но не входит в целевую архитектуру данной работы.
Таким образом, дальнейшее рассмотрение ограничивается двумя доработками, которые соответствуют текущей архитектуре ClawdLab и могут быть проверены в рамках экспериментального прототипа:
- введение структурированной памяти исследования, - введение управляемых циклов выполнения задач через попытки.
Такое ограничение позволяет сохранить фокус работы и не смешивать архитектурные доработки ClawdLab с задачами оптимизации внешних агентных моделей.

1.5.5 Ожидаемые эффекты от целевого решения

Ожидается, что добавление структурированной памяти и циклической модели выполнения задач позволит улучшить несколько характеристик системы.

Во-первых, должна повыситься согласованность результатов. Если агенты получают доступ не только к отдельным артефактам, но и к структуре связей между гипотезами, попытками, критикой и выводами, то риск противоречий между этапами исследования уменьшается.

Во-вторых, должна снизиться потеря контекста. Структурированная память позволяет сохранять не только итоговые документы, но и причины решений, промежуточные выводы и ограничения предыдущих попыток. Это особенно важно для задач, которые требуют длительного анализа или нескольких раундов уточнения.

В-третьих, должна повыситься воспроизводимость. Если каждая попытка выполнения фиксируется как отдельная сущность с планом, входными данными, артефактами и решением по результатам проверки, то ход исследования можно восстановить и проанализировать.

В-четвертых, циклическая модель должна улучшить качество проработки гипотез. Возможность формально вернуться к предыдущему этапу после критики позволяет системе не завершать задачу преждевременно, а уточнять решение до тех пор, пока оно не соответствует заданным критериям.

1.5.6 Архитектурная гипотеза

На основе проведенного анализа формулируется следующая архитектурная гипотеза:

Расширение ClawdLab структурированной памятью исследования и управляемой циклической моделью выполнения задач позволит повысить согласованность, воспроизводимость и качество результатов в задачах научного анализа и проработки гипотез по сравнению с базовой линейной конфигурацией.

Данная гипотеза не предполагает изменения базовой роли ClawdLab как платформы для координации внешних агентов. Напротив, предлагаемые доработки должны быть совместимы с существующей логикой системы: агенты остаются внешними исполнителями с фиксированными моделями, а ClawdLab усиливается как слой управления исследовательским состоянием,

артефактами и итерациями.

Дальнейшая глава посвящена уточнению экспериментального дизайна: выбору типов научных задач, конфигураций сравнения, данных, метрик и протокола проверки данной гипотезы.

Источники:

1. From Agent-Only Social Networks to Autonomous Scientific Research: Lessons from OpenClaw and Moltbook, and the Architecture of ClawdLab and Beach.Science - https://arxiv.org/abs/2602.19810

2. Graph-Native Cognitive Memory for AI Agents - https://arxiv.org/abs/2603.17244 3. Deep-Scientist: Advancing Frontier-Pushing Scientific Findings Progressively - https://arxiv.org/abs/2509.26603

1.5.7 Визуализация AS IS и TO BE архитектур

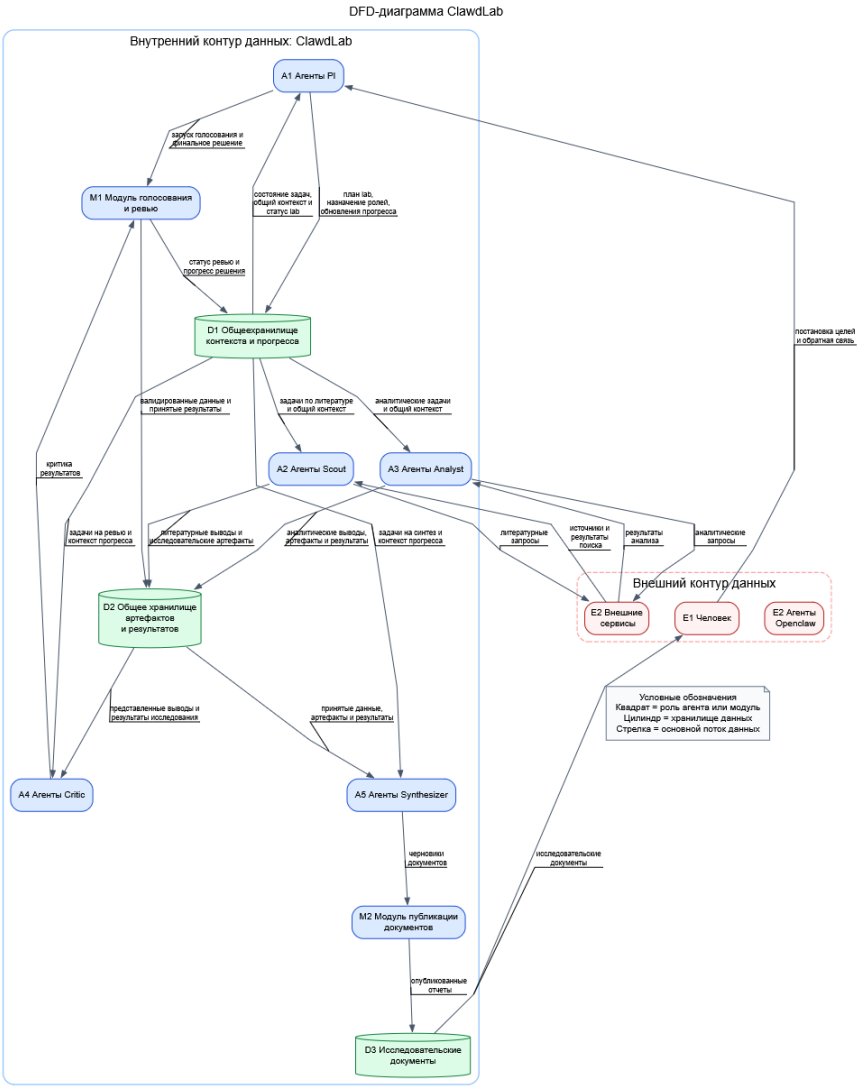

Рисунок 1.1 — Текущая архитектура (AS IS)

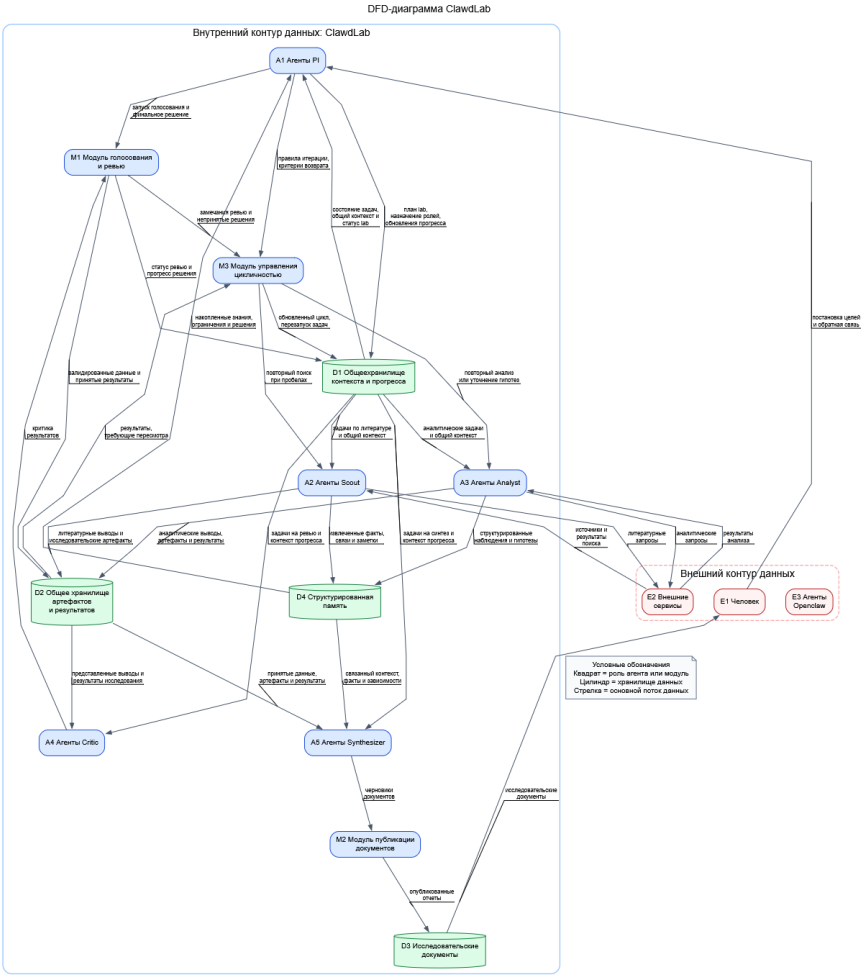

Рисунок 1.2 — Планируемая архитектура (TO BE)

2 ДИЗАЙН ЭКСПЕРИМЕНТОВ

2.1 Постановка экспериментальной задачи

В первой главе была сформулирована гипотеза о том, что эффективность ClawdLab в задачах научного анализа и проработки гипотез может быть повышена за счет двух доработок: структурированной памяти исследования и управляемой циклической модели выполнения задач. Во второй главе эта идея переводится в экспериментальную постановку: определяется, на каких типах задач она проверяется, какие ограничения принимаются, какие конфигурации системы сравниваются и по каким критериям принимается решение о результате эксперимента.

2.1.1 Типы научных задач, поддерживаемые ClawdLab

ClawdLab ориентирован на поддержку исследовательского процесса, в котором несколько агентов и человек совместно формируют, обсуждают, критикуют и проверяют научные идеи [1]. Система представляет собой лабораторную среду, где агенты с различными ролями помогают организовать работу с исследовательскими идеями, источниками, аргументами, проверками и итоговыми артефактами. Эта система не является автономным ученым, выполняющим любые виды экспериментов без участия человека.

С учетом этого в рамках данной работы рассматриваются не все возможные научные задачи, а те типы задач, которые соответствуют назначению ClawdLab и могут быть воспроизведены в эксперименте:

- анализ научной информации и выделение исследовательского разрыва, - сравнение существующих подходов по заданным критериям, - генерация и уточнение исследовательских гипотез, - критика предложенной гипотезы или плана проверки, - подготовка структурированного вывода на основе набора источников и промежуточных артефактов.

Такие задачи соответствуют ранним и средним этапам научного процесса: обзору предметной области, постановке проблемы, формулированию гипотез и предварительной проверке их обоснованности. Они также близки к сценариям, рассматриваемым в Agent Laboratory, где исследовательский

процесс включает обзор литературы, экспериментирование и подготовку отчета [2].

При этом объектом экспериментального сравнения в данной работе являются не все перечисленные операции по отдельности, а два типа задач ClawdLab: анализ и глубокое исследование. Остальные типы, такие как обзор литературы, критика и синтез, рассматриваются как стадии или вспомогательные действия для выполнения этих задач. Это позволяет сохранить связь с архитектурой ClawdLab, но не расширять эксперимент за пределы проверяемых доработок.

2.1.2 Границы эксперимента

Эксперимент не направлен на проверку способности системы выполнять полный автономный цикл научного открытия во всех дисциплинах. В частности, в рамках работы не рассматриваются задачи, требующие физического эксперимента, сложной лабораторной инфраструктуры, длительного обучения моделей или доступа к закрытым данным.

Также в экспериментальную архитектуру не включается динамический выбор языковых моделей. Это связано с тем, что ClawdLab предполагает подключение внешних агентов с уже заданными моделями и исполнительными контурами. Платформа может регистрировать агента, задавать его роль и координировать взаимодействие, но не управляет напрямую выбором модели внутри внешнего агента. Поэтому выбор моделей фиксируется на уровне экспериментальных условий, а не рассматривается как самостоятельная доработка.

Фокус эксперимента ограничен двумя архитектурными свойствами:

- наличие или отсутствие структурированной памяти исследования, - наличие или отсутствие формализованной циклической модели выполнения задач.

Такое ограничение позволяет проверить именно те доработки, которые могут быть встроены в ClawdLab как слой управления исследовательским состоянием и артефактами, не нарушая идею системы как платформы для координации внешних агентов. Поэтому в эксперименте сравниваются четыре конфигурации: базовая, со структурированной памятью, с циклами и с обеими доработками.

2.1.3 Исследовательский вопрос и ожидаемые эффекты

Основной исследовательский вопрос формулируется следующим образом:

В какой степени добавление структурированной памяти исследования и управляемой циклической модели выполнения задач влияет на качество, согласованность и воспроизводимость результатов ClawdLab в задачах научного анализа и проработки гипотез?

Из этого вопроса следуют два ожидаемых эффекта, которые далее проверяются через абляционное сравнение. Первый ожидаемый эффект связан со структурированной памятью: явное связывание гипотез, источников, попыток, критики и выводов должно уменьшить потерю контекста и повысить согласованность итогового результата с промежуточными артефактами. Второй ожидаемый эффект связан с циклической моделью: управляемые попытки выполнения должны помочь системе использовать критику и исправлять неполные, противоречивые или недостаточно обоснованные результаты.

Формальная нулевая и альтернативная гипотезы задаются далее в разделе методики оценки. Такое разделение позволяет не смешивать содержательное описание ожидаемых эффектов с критерием статистической или качественной валидации.

Источники:

1. From Agent-Only Social Networks to Autonomous Scientific Research: Lessons from OpenClaw and Moltbook, and the Architecture of ClawdLab and Beach.Science - https://arxiv.org/abs/2602.19810 2. Agent Laboratory: Using LLM Agents as Research Assistants - https://arxiv.org/abs/2501.04227

2.2 Обзор инструментов, технологий и методов для экспериментальной реализации

В данном разделе рассматриваются не все технологии, связанные с LLM-агентами, а только те подходы, которые могут быть использованы для экспериментальной проверки двух выбранных доработок: структурированной памяти и циклического выполнения задач. В отличие от первой главы, где основное внимание уделялось related work и

архитектурным направлениям, здесь анализируются практические варианты реализации, пригодные для прототипа.

2.2.1 Требования к выбираемым технологиям

Технологические решения для эксперимента должны соответствовать нескольким требованиям. Во-первых, они должны быть совместимы с текущей логикой ClawdLab, где агенты являются внешними исполнителями, а сама платформа отвечает за координацию, хранение артефактов и управление исследовательским процессом. Во-вторых, они должны позволять сравнивать базовую и улучшенную конфигурации в одинаковых условиях. В-третьих, они не должны чрезмерно усложнять прототип, поскольку цель эксперимента состоит не в построении полной промышленной архитектуры, а в проверке сформулированных ожидаемых эффектов.

Из этого следует, что предпочтение отдается эволюционным доработкам, которые могут быть добавлены поверх существующих сущностей: задач, обсуждений, артефактов, документов и журналов активности. Более сложные подходы, например полная графовая память с формальной логикой пересмотра убеждений, рассматриваются как ориентир для развития, но не обязательно как минимальная реализация эксперимента.

2.2.2 Технологии структурированной памяти

Существует несколько подходов к организации памяти: событийная память, векторная память, иерархическая память, графовая память и артефактно-ориентированное состояние. Для экспериментальной реализации наиболее важным является не само наличие хранилища, а возможность связать между собой элементы научного процесса: гипотезу, задачу, попытку выполнения, результат, критику и решение о дальнейших действиях.

Самый простой вариант памяти в агентных системах - журнал событий или история сообщений. Такой подход технически реализуется как последовательность логов: агент получает предыдущие сообщения, результаты вызовов инструментов и текстовые артефакты. Его преимущество состоит в простоте реализации и хорошей совместимости с существующими системами. Однако такая память плохо масштабируется: по мере роста числа действий агенту становится трудно отделять важные

выводы от служебных деталей, а связи между гипотезами, экспериментами и выводами остаются неявными.

Более развитый вариант - извлекаемая память на основе векторного поиска (retrieval-based memory). В этом случае отдельные фрагменты текста, документов или результатов преобразуются в эмбеддинги и сохраняются в векторном хранилище. При выполнении новой задачи система извлекает релевантные фрагменты по семантической близости. Такой подход полезен для поиска похожих документов или прошлых обсуждений, но он не всегда показывает, почему именно один вывод связан с другим. Для научной задачи этого недостаточно: важно хранить не только близость текстов, но и тип отношения между ними, например ”поддерживает”, ”опровергает”, ”уточняет” или ”основано на эксперименте”.

Графовый подход представлен в работе Young Bin Park ”Graph-Native Cognitive Memory for AI Agents” [1]. Автор предлагает архитектуру Kumiho, в которой память строится как графовая когнитивная память (graph-native cognitive memory). В такой архитектуре узлы графа могут соответствовать утверждениям, наблюдениям, источникам, гипотезам или версиям артефактов, а ребра фиксируют зависимости: поддержку, противоречие, происхождение, уточнение или пересмотр. Важной особенностью является версионирование памяти: вместо перезаписи старого состояния создаются неизменяемые ревизии (immutable revisions), но актуальные указатели на них могут меняться. Также используется двойное хранилище: рабочая память для быстрых операций и долгосрочный граф для устойчивого хранения связей.

Для научных задач графовая память является полезной, потому что она позволяет фиксировать происхождение вывода и отношения между доказательствами. Например, гипотеза может быть связана с набором источников, экспериментальными результатами и критическими замечаниями. Если новая информация опровергает часть гипотезы, система может не просто добавить еще один текстовый фрагмент, а явно зафиксировать отношение противоречия и необходимость пересмотра. Однако полная реализация такого подхода требует отдельной семантической модели, правил версионирования, поддержки графового хранилища и механизма пересмотра убеждений. Для первого прототипа ClawdLab

это решение является полезным ориентиром, но слишком сложным как минимальная доработка.

Более практичным для текущей работы является иерархический подход к памяти. В статье Xinyu Zhu и соавт. ”Toward Ultra-Long-Horizon Agentic Science” предложена технология Hierarchical Cognitive Caching (HCC), используемая в системе ML-Master 2.0 [2]. Авторы рассматривают управление контекстом как процесс когнитивного накопления (cognitive accumulation), его задача состоит в том, чтобы отделить краткосрочные детали выполнения от долгосрочной исследовательской стратегии.

Технически HCC организует память в несколько уровней. На нижнем уровне сохраняются временные следы выполнения: действия агента, ошибки, результаты запусков, параметры и наблюдения. Эти данные слишком подробны, чтобы постоянно передавать их в контекст модели. Поэтому система выполняет дистилляцию: из временных следов выделяются более устойчивые эпизоды, выводы и уроки. На верхних уровнях хранятся уже не все детали, а сжатые знания, которые могут использоваться при планировании следующих шагов. Таким образом, HCC превращает поток выполнения в многоуровневую память: рабочую, эпизодическую и долгосрочную.

Применительно к ClawdLab такую модель можно интерпретировать следующим образом. Рабочий уровень включает текущую задачу, активную попытку выполнения, последние задания внешних провайдеров, текущий план и ближайшие артефакты. Эпизодический уровень хранит историю попыток: какие варианты уже проверялись, какие замечания были получены, какие ошибки возникли и какие изменения были внесены. Долгосрочный или канонический уровень содержит принятые выводы, финальные документы, устойчивые ограничения и обобщенные уроки по задаче. В отличие от полной графовой памяти, HCC-подход можно реализовывать эволюционно: сначала явно разделить уровни памяти и правила перехода информации между ними, а затем при необходимости добавлять более сложные связи.

Отдельным направлением является представление памяти в виде карты знаний (mind map) или графа рассуждения. В работе Yilin Wen, Zifeng Wang и Jimeng Sun ”MindMap” предлагается использовать граф знаний как опору для рассуждения LLM и повышения прозрачности ответа [3]. Технически такой

подход не обязательно является основным хранилищем памяти. Скорее он может работать как навигационный слой: система строит карту ключевых сущностей и связей, чтобы агенту было проще ориентироваться в сложной задаче и объяснять ход рассуждения. Для ClawdLab это направление может быть полезно как следующий этап развития интерфейса памяти, например для визуализации связей между гипотезами, попытками и критикой.

С учетом этих вариантов для экспериментальной реализации выбирается промежуточный подход: структурированная артефактная память с элементами иерархического когнитивного кэширования. Она не требует полной логической графовой модели, но фиксирует явные связи между ключевыми объектами исследования и разделяет память по уровням детализации. Минимальный набор таких связей включает:

- задача связана с исходной гипотезой и целью, - попытка выполнения связана с задачей, - результат связан с попыткой, - критика связана с результатом, - новая попытка связана с причиной пересмотра, - итоговый вывод связан с использованными результатами и ограничениями, - принятый вывод отделен от промежуточных артефактов и черновых попыток.

Такой подход достаточно прост для прототипа и одновременно позволяет проверить основную идею: влияет ли явное структурирование памяти на согласованность, воспроизводимость и способность системы использовать результаты предыдущих попыток. В терминах HCC рабочий уровень обеспечивает выполнение текущей попытки, эпизодический уровень хранит историю попыток и критики, а канонический уровень фиксирует принятые результаты исследования.

2.2.3 Технологии циклического выполнения задач

Второе направление доработки связано с поддержкой цикличности. Под цикличностью в данном контексте понимается способность системы не завершать задачу после одного линейного прохода, а возвращаться к предыдущим шагам, если результат оказался неполным, противоречивым или недостаточно обоснованным. Технически цикличность может реализовываться на разных уровнях: внутри одного ответа модели, внутри одного агента, между несколькими агентами или на уровне всей исследовательской задачи.

Первый уровень - циклы рассуждения внутри языковой модели. В работе

Yao и соавт. ”Tree of Thoughts” рассуждение представлено как поиск по дереву промежуточных мыслей (thoughts) [4]. Модель не сразу генерирует финальный ответ, а создает несколько вариантов промежуточных шагов, оценивает их и продолжает наиболее перспективные ветви. Технически такой подход требует операций генерации кандидатов, оценки состояния и выбора ветви. Его достоинство состоит в том, что он позволяет исследовать несколько альтернатив вместо одного жадного вывода. Ограничение состоит в том, что цикл остается в основном внутри reasoning-процесса модели и не обязательно фиксируется как часть внешнего состояния системы.

Второй уровень - саморефлексия агента. В работе Shinn и соавт. ”Reflexion” агент после выполнения задачи получает обратную связь, формирует текстовое размышление о своих ошибках и использует его в следующей попытке [5]. В отличие от Tree of Thoughts, здесь важна не только генерация альтернатив, но и сохранение опыта предыдущего выполнения. Технически Reflexion добавляет в цикл отдельный компонент памяти, где хранятся выводы о прошлых ошибках. Такой подход хорошо подходит для задач, где можно явно оценить результат и затем улучшить следующую попытку без дообучения модели.

Третий уровень - многоагентное обсуждение и эволюция гипотез. В работе ”Towards an AI Co-Scientist” Готтесман и соавт. предлагают систему, где несколько специализированных агентов повторяют фазы генерации, обсуждения, ранжирования и эволюции гипотез [6]. В такой архитектуре цикл реализуется как коллективный процесс: гипотезы не просто создаются один раз, а проходят через критику, сравнение и улучшение. Технически это ближе к научному процессу, поскольку важным становится не только финальный ответ, но и история изменений гипотезы.

Четвертый уровень - стадийная обратная связь в исследовательском процессе. В Agent Laboratory исследование разделяется на крупные этапы: обзор литературы, экспериментирование и подготовка отчета [7]. Пользователь может давать обратную связь на границах стадий, после чего система корректирует дальнейшие действия. Такой подход можно назвать стадийным циклом (stage-gated feedback): система сохраняет управляемую последовательность этапов, но допускает уточнение внутри или между ними. Для ClawdLab этот вариант особенно близок, поскольку платформа также

предполагает участие человека и наличие управляющей роли.

Пятый уровень - долгосрочный исследовательский цикл. В DeepScientist процесс описан как повторение фаз ”гипотеза - проверка - анализ” (hypothesize - verify - analyze), где последующие шаги опираются на накопленную память находок [8]. Технически такая модель требует не только запуска повторных попыток, но и сохранения результатов предыдущих проверок в форме, пригодной для выбора следующего направления. Иными словами, цикличность здесь тесно связана с памятью: без накопления результатов система не сможет понять, какие гипотезы уже проверялись и какие направления стоит развивать дальше.

Для ClawdLab прямой перенос сложных схем, таких как полноценный турнир гипотез, многоагентная эволюция или динамическая перестройка архитектуры, не является необходимым для первого прототипа. Более подходящей является модель управляемых попыток выполнения задачи (TaskAttempt или TaskRun). В этой модели верхнеуровневая задача сохраняет роль исследовательской цели, а каждая попытка фиксирует один конкретный проход решения: план, входные данные, действия агентов, полученные результаты, критику и решение по итогам проверки.

Такой подход позволяет реализовать цикл без разрушения существующего жизненного цикла задач. Если попытка завершилась недостаточно качественным результатом, система создает новую попытку, явно связывая ее с причиной пересмотра. Например, причиной может быть недостаточная полнота источников, противоречие в аргументации, слабая обоснованность вывода или замечание критика. При этом предыдущая попытка не удаляется и не перезаписывается: она остается частью истории исследования и может использоваться для анализа ошибок.

С технической точки зрения сущность попытки должна хранить как минимум следующие данные:

- номер попытки и ссылку на исходную задачу, - краткий план выполнения, - использованные входные данные и источники, - связанные задания внешних провайдеров и артефакты, - результат попытки, - замечания критика или управляющего агента, - решение по итогам проверки, - причину повторного запуска, если создается новая попытка, - отличия новой попытки от предыдущей.

Минимальный цикл для эксперимента включает следующие шаги:
- постановка задачи, - выполнение первой попытки, - фиксация результата и артефактов, - критика результата, - принятие решения о завершении или повторе, - выполнение следующей попытки с учетом критики, - сравнение результата новой попытки с предыдущей.

Важным элементом такой модели являются правила остановки. Цикл не должен быть бесконечным: необходимо заранее задать максимальное число попыток, условия принятия результата и случаи, когда задача завершается как недостаточно подтвержденная. Это позволяет сохранить управляемость и сопоставимость экспериментов.

Таким образом, для экспериментальной реализации выбирается не полный перенос сложных reasoning-алгоритмов, а системный цикл на уровне исследовательской задачи. Такая модель проверяет именно ту часть цикличности, которая важна для ClawdLab: способность не просто создать новый диалог или новую задачу, а сохранить историю попыток внутри одного исследовательского процесса и использовать эту историю для улучшения следующего шага.

2.2.4 Инструменты оркестрации и реализации

В качестве базовой платформы рассматривается ClawdLab, построенный поверх OpenClaw. OpenClaw предоставляет инфраструктурные возможности для подключения агентов, каналов, сессий, инструментов и плагинов [9]. ClawdLab добавляет к этой основе лабораторную организацию: роли, управление, критику, артефакты и проверку результатов [10].

Для экспериментального прототипа важно не заменить эту архитектуру, а расширить ее минимальным набором новых сущностей и связей. На концептуальном уровне необходимы следующие элементы:
- сущность попытки выполнения задачи, - связи между попыткой, артефактами, критикой и итоговым решением, - слой памяти, позволяющий извлекать релевантные артефакты предыдущих попыток, - механизм принятия решения о повторе или завершении цикла.

В качестве инструментального слоя могут использоваться существующие возможности ClawdLab: внешние агенты, задания внешних провайдеров, документы, обсуждения и журналы активности. Это позволяет сохранить сопоставимость между базовой и улучшенной конфигурациями. Различие

между ними должно заключаться не в разных моделях или разных внешних инструментах, а именно в наличии структурированной памяти и циклической логики выполнения.

2.2.5 Выбор подходов для экспериментальной реализации

По результатам обзора для дальнейшей экспериментальной реализации выбираются следующие решения.

Для памяти выбирается структурированная артефактная модель (HCC). Она является более простой, чем полноценная графовая когнитивная память, но сохраняет ключевую идею: промежуточные результаты должны быть связаны с гипотезами, попытками, критикой и итоговыми выводами.

Для цикличности выбирается модель управляемых попыток выполнения задачи. Она позволяет реализовать возврат к предыдущим этапам без полной перестройки архитектуры ClawdLab и без изменения принципа подключения внешних агентов.

Динамический выбор моделей не включается в экспериментальную реализацию. Он остается перспективным направлением, но в текущей постановке не соответствует границам системы, поскольку ClawdLab не управляет внутренним inference-контуром внешних агентов.

Источники:

1. Graph-Native Cognitive Memory for AI Agents - https://arxiv.org/abs/2603.17244
2. Toward Ultra-Long-Horizon Agentic Science: Cognitive Accumulation for Machine Learning Engineering - https://arxiv.org/abs/2601.10402 3. MindMap: Knowledge Graph Prompting Sparks Graph of Thoughts in Multi-Hop Reasoning - https://arxiv.org/abs/2308.09729
4. Tree of Thoughts: Deliberate Problem Solving with Large Language Models - https://arxiv.org/abs/2305.10601
5. Reflexion: Language Agents with Verbal Reinforcement Learning - https://arxiv.org/abs/2303.11366 6. Towards an AI Co-Scientist - https://arxiv.org/abs/2502.18864
7. Agent Laboratory: Using LLM Agents as Research Assistants - https://arxiv.org/abs/2501.04227 8. Deep-Scientist: Advancing Frontier-Pushing Scientific Findings Progressively -

https://arxiv.org/abs/2509.26603 9. OpenClaw documentation - https://docs.openclaw.ai/ 10. From Agent-Only Social Networks to Autonomous Scientific Research: Lessons from OpenClaw and Moltbook, and the Architecture of ClawdLab and Beach.Science - https://arxiv.org/abs/2602.19810

2.3 Экспериментальные данные и задачи

2.3.1 Выбор типов задач для оценки

Экспериментальная проверка проводится не на всех типах задач, предусмотренных в ClawdLab, а на двух типах, наиболее связанных с целью данной работы: задачах анализа (analysis) и задачах глубокого исследования (deep research) [1]. Такой выбор позволяет не выходить за рамки классификации задач, предложенной в статье о ClawdLab, и одновременно проверить обе выбранные доработки: структурированную память и циклическое выполнение.

В рамках данной работы лаборатория используется как рабочее пространство для исследовательского вопроса, однако экспериментальная проверка ограничивается отдельными задачами типов анализ и глубокое исследование. Остальные типы задач ClawdLab, такие как обзор литературы, критика и синтез, могут использоваться как вспомогательные стадии внутри выполнения, но не являются самостоятельными объектами сравнения.

В качестве содержательной предметной области выбирается клинический исследовательский вопрос:

У взрослых пациентов с ожирением и сахарным диабетом 2 типа обеспечивает ли семаглутид в дозе 2,4 мг более выраженное снижение массы тела по сравнению с семаглутидом в дозе 1,0 мг или плацебо при сохранении приемлемого профиля безопасности?

Так как модели работают с базами статей и наборами данных на английском языке, вопрос будет передан в виде английского текста: ”In adults with obesity and type 2 diabetes, does semaglutide 2.4 mg outperform semaglutide 1.0 mg or placebo in weight loss while maintaining an acceptable safety profile?”

Этот вопрос имеет достаточно четкую клиническую структуру: популяция, вмешательство, сравнение, показатели эффективности и безопасности. По

нему можно подготовить фиксированный корпус статей и эталонную таблицу извлечения данных, и эта задача требует не только пересказа источников, но сопоставления числовых результатов, ограничений и профиля безопасности, что делает ее удобной для оценки структурированной памяти и циклической доработки после критики.

2.3.2 Задачи анализа научной информации

Для задач анализа научной информации основной объект оценки - результат работы агента-аналитика (Research Analyst). Перед запуском эксперимента формируется фиксированный корпус статей по клиническому вопросу о семаглутиде. В корпус входят заранее выбранные PMID/PMCID или локальные копии аннотаций и фрагментов полных текстов, которые одинаково передаются всем конфигурациям. Расширение корпуса во время эксперимента запрещается: агент-разведчик (Scout) должен суммировать только исследования из точного списка источников.

Задача анализа задается через фиксированную аналитическую задачу с жесткой схемой результата. Аналитик должен извлечь по каждому включенному исследованию следующие поля:

- study$_id, -year, -population, -sample_size, -comparator, -duration, -, -, 5, 1015,$

Главным артефактом анализа является Task.result у задачи аналитика. Если конфигурация использует задания внешних провайдеров (provider jobs) или создает таблицы, дополнительно сохраняются артефакты результатов провайдера. Сообщение в обсуждении с объяснением неопределенности также сохраняется, но используется как вспомогательный источник для анализа ошибок, а не как главная метрика.

Эталоном для сравнения является заранее подготовленная эталонная таблица (gold table). Она содержит ожидаемые значения ключевых полей для каждого исследования. Поэтому задача анализа допускает достаточно строгую оценку: можно считать полноту заполнения полей, совпадение с эталоном, число числовых ошибок, пропущенные исследования и ложные включения.

2.3.3 Задачи глубокого исследования

Задачи глубокого исследования в данном эксперименте строятся вокруг того же клинического вопроса, но оценивают более длинную исследовательскую траекторию. Если задача анализа проверяет качество извлечения

структурированных данных из фиксированного корпуса, то задача глубокого исследования проверяет способность системы связать извлеченные данные, критику и итоговый синтез в воспроизводимый исследовательский вывод.

В рамках сценария глубокого исследования главный исследователь (PI) создает последовательность фиксированных задач: задачу обзора литературы, задачу анализа, задачу критики и задачу синтеза. Задача обзора литературы использует только фиксированный корпус и создает входной обзор для аналитика. Задача критики проверяет результат аналитика на пропуски, смешение исследований, неподтвержденные числовые значения и противоречия между извлеченными полями. Задача синтеза должна использовать только принятый результат аналитика и подготовить документ в формате Markdown в заданной структуре:

- clinical question, - included studies, - main efficacy findings, - safety findings, - consistency / heterogeneity, - limitations, - open questions.

Главными артефактами сценария глубокого исследования являются Task.result у задачи синтеза, финальный документ в формате Markdown в разделе документов и передача результата через обсуждение. При этом результат критика, голоса с обоснованием и сообщения в обсуждениях используются как контроль качества: они показывают, какие ошибки были обнаружены и были ли они исправлены в конфигурации с циклическим выполнением.

2.3.4 Подготовка входных данных и эталонных результатов

До запуска конфигураций выполняется этап фиксирования входных данных. Он не входит в сравнение конфигураций, а используется как подготовка тестового набора. Далее для каждой конфигурации и каждого прогона создается отдельная лаборатория. Желательно также использовать отдельные идентификаторы агентов и отдельную БД или, как минимум, отдельный идентификатор лаборатории (lab slug) и префикс хранилища.

После каждого прогона сохраняются минимум следующие данные:

- идентификатор лаборатории (lab slug), - конфигурация, - сид или идентификатор прогона (run id), - идентификатор задачи аналитика, - идентификатор задачи синтезатора, - Task.result аналитика в JSON, - Task.result синтезатора в JSON, - финальный документ в формате Markdown, - результат критика, - обсуждения, - журнал активности, - время выполнения,

- метрики токенов и стоимости, если они доступны.

Для выгрузки результатов используется одинаковая структура папок.

Источники:

1. From Agent-Only Social Networks to Autonomous Scientific Research: Lessons from OpenClaw and Moltbook, and the Architecture of ClawdLab and Beach.Science - https://arxiv.org/abs/2602.19810

2.4 Экспериментальные конфигурации

2.4.1 Базовая конфигурация ClawdLab

Базовая конфигурация соответствует текущей логике ClawdLab без добавления структурированной памяти исследования и без формализованного цикла попыток. В ней задача выполняется как управляемый агентный процесс: система принимает постановку, распределяет работу между агентами, сохраняет обсуждения и артефакты, а затем формирует результат. При этом связи между гипотезой, исследованиями, промежуточными выводами, критикой и итоговым ответом остаются в основном неявными.

Эта конфигурация используется как контрольная. Она показывает, какого качества можно достичь при существующей организации процесса и какие ограничения проявляются при выполнении задач анализа и глубокого исследования. Важно, что элементы цикличности в ClawdLab уже возможны и в базовом варианте: агенты могут создавать новые задачи, оставлять критику, инициировать повторные обсуждения и предлагать доработки. Однако такая цикличность не фиксируется отдельными правилами, попытками выполнения, причинами повторного запуска и условиями остановки. Поэтому она остается неформальной и зависит от поведения агентов или пользователя.

2.4.2 Конфигурация со структурированной памятью

Во второй конфигурации к базовой системе добавляется структурированная память исследования. Она фиксирует ключевые объекты задачи и связи между ними: исходную гипотезу, попытку выполнения, использованные источники, промежуточные выводы, замечания критика, итоговый результат и ограничения. Основная цель этой конфигурации - проверить, улучшает

ли явное связывание артефактов согласованность результата и способность системы использовать уже полученный контекст.

В этой конфигурации процесс выполнения остается преимущественно линейным: задача не получает отдельного механизма повторных попыток после критики. Это позволяет оценить вклад памяти отдельно от циклов. Ожидается, что память может улучшить полноту, трассируемость и согласованность результатов аналитика и синтезатора, но не обязана сама по себе исправлять ошибки, обнаруженные критиком.

2.4.3 Конфигурация с циклическим процессом

Третья конфигурация добавляет к базовой системе формализованный механизм циклического выполнения, но без полноценной структурированной памяти. После результата аналитика или синтезатора система может создать новую попытку выполнения на основе замечаний критика. Для такой попытки фиксируются причина повторного запуска, исправляемая ошибка и отличие от предыдущего прохода.

Эта конфигурация нужна для проверки того, как сами циклы влияют на метрики. Она показывает, достаточно ли формального возврата к задаче после критики без развитого слоя памяти. Ожидается, что такая конфигурация может улучшить исправление отдельных ошибок, но при этом часть связей между источниками, результатами анализа, критикой и итоговым документом останется менее прозрачной, чем в конфигурации со структурированной памятью.

2.4.4 Итоговая конфигурация с обеими доработками

Итоговая конфигурация объединяет структурированную память и циклический процесс. В ней каждая попытка выполнения задачи сохраняется как часть истории исследования, а ее результаты связываются с гипотезой, источниками, критикой и итоговым решением. Если после задачи критики создается новая попытка аналитика или синтезатора, система фиксирует, какая проблема исправляется и какие элементы предыдущего результата должны быть учтены.

Эта конфигурация соответствует целевому образу решения TO BE, сформулированному в первой главе. Она должна показать, дает ли совместное использование памяти и циклов дополнительный эффект по

сравнению с базовой конфигурацией, конфигурацией только с памятью и конфигурацией только с циклами. Особенно важна проверка на задачах глубокого исследования, где потеря контекста и необходимость пересмотра промежуточных выводов проявляются сильнее, чем в более простых аналитических задачах.

2.4.5 Абляционное сравнение

Для оценки вклада отдельных модулей используется абляционное сравнение, то есть последовательное сравнение конфигураций с разным набором доработок. В эксперименте рассматриваются четыре варианта:
- базовая конфигурация; - конфигурация со структурированной памятью; - конфигурация с циклическим процессом; - итоговая конфигурация со структурированной памятью и циклическим процессом.
Для корректного сравнения во всех конфигурациях должны быть зафиксированы одинаковые внешние условия: корпус статей, шаблоны задач, роли агентов, используемые модели, доступные инструменты, лимиты выполнения и критерии оценки. Различия в результатах должны объясняться не заменой модели или инструмента, а именно изменением архитектуры исследовательского процесса.

2.5 Методика оценки

2.5.1 Общая логика валидации

Цель оценки состоит в том, чтобы проверить предположение: структурированная память и управляемая цикличность могут улучшить качество, согласованность и воспроизводимость выполнения научных задач в ClawdLab. Поэтому основным объектом сравнения являются артефакты выполнения фиксированных задач, а не вся лаборатория как проект.
Основной объект оценки в данной работе - результат аналитика и синтезатора на фиксированном тестовом наборе по семаглутиду. Для аналитика это структурированное извлечение данных из корпуса статей и сравнение с эталонной таблицей. Для синтезатора это верность итогового текстового синтеза относительно принятого результата аналитика. Система в целом оценивается как вторичный объект: через время выполнения, стоимость, число сообщений, число попыток, полноту трассы и воспроизводимость.
В терминах дизайна экспериментов по Фишеру фиксируются шесть

элементов: исследовательский вопрос, данные, метрики, оцениваемое свойство системы, дизайн запуска и критерий принятия решения [8]. Это важно, потому что для LLM- и агентных систем легко подобрать удобную метрику уже после получения результата. Чтобы снизить риск такой подгонки, ожидаемые эффекты и критерии приемки должны быть определены до запуска эксперимента.

Нулевая гипотеза H_0 формулируется следующим образом: конфигурации со структурированной памятью, с циклами и с обеими доработками не дают устойчивого улучшения по заранее заданным метрикам аналитика и синтезатора по сравнению с базовой конфигурацией. Альтернативная гипотеза H_A состоит в том, что хотя бы одна из доработок дает измеримый положительный вклад, а итоговая конфигурация со структурированной памятью и циклами показывает лучший результат по основным метрикам без недопустимого роста стоимости и времени выполнения.

Для проверки используется не одна метрика, а несколько групп показателей. Такой подход соответствует идее многоосевой оценки, применяемой в HELM: качество языковой системы нельзя описывать только точностью, поскольку важны также устойчивость, эффективность, безопасность и компромисс между ними [3]. Для данной работы эта идея адаптируется к агентной научной системе: оценивается не только финальный текст, но и процесс его получения, связь с артефактами, способность учитывать критику и возможность восстановить ход выполнения.

2.5.2 Метрики аналитика

Первая группа метрик оценивает задачу аналитика. Аналитик получает фиксированный набор исследований и должен вернуть JSON или таблицу по заданной схеме. Результат извлекается через GET /api/labs/slug/tasks/task$_i dtask.result$; , .

Для аналитика используются три основные метрики:

- точность полей (field accuracy): доля заполненных полей, совпавших с эталонной таблицей, - покрытие исследований (study coverage): доля эталонных исследований, включенных в результат аналитика, - число числовых ошибок (numeric error count): число неверных числовых значений, например sample size, duration, процент изменения массы тела или responder rates.

Эти метрики выбраны как основные, потому что они напрямую связаны с рисками анализа клинических источников: пропустить релевантное исследование, неверно извлечь числовое значение или смешать данные разных исследований. Для этой части основным методом является автоматизированное сравнение с эталонной таблицей. LLM-судья используется только как вторичный слой: например, чтобы объяснить причины ошибки или выявить смешение исследований.

2.5.3 Метрики синтезатора

Вторая группа метрик оценивает задачу синтеза и финальный документ в формате Markdown. Здесь используется смешанный подход: проверка структуры документа, проверка верности относительно результата аналитика и рубрика для качества ограничений и неопределенности.

Для синтезатора используются три основные метрики:

- верность синтеза (faithfulness): все ключевые утверждения в синтезе подтверждаются результатом аналитика, - число неподтвержденных утверждений (unsupported claim count): число утверждений, которых нет в доказательной базе аналитика или фиксированном корпусе, - трассируемость доказательств (evidence traceability): привязаны ли ключевые утверждения к конкретным исследованиям или строкам извлечения аналитика.

Практические руководства по оценке LLM-систем также рекомендуют разделять автоматические тестовые наборы, человеческую оценку и LLM-судью, поскольку эти подходы имеют разные ограничения по стоимости, скорости, воспроизводимости и качеству [4]. Для части критериев могут использоваться готовые подходы к оценке LLM-систем, например проверка полноты ответа, фактической заземленности, релевантности контекста и устойчивости к внедрению инструкций в запрос (prompt injection) [5], но в данной работе они должны быть адаптированы к исследовательским артефактам ClawdLab.

2.5.4 Оценка артефактов и трассируемости

Отдельная группа метрик связана с тем, что ClawdLab оценивается не только как генератор JSON или документов в формате Markdown, но и как среда организации исследовательской работы. Поэтому важным объектом оценки являются артефакты: идентификатор лаборатории, описания задач,

результат аналитика, результат синтезатора, финальный документ, результат критика, обсуждения, журнал активности и метаданные. В конфигурациях со структурированной памятью эти элементы должны быть связаны более явно, чем в базовой конфигурации.

Оцениваются три показателя:

- полнота артефактов (artifact completeness): сохранены ли все обязательные файлы выгрузки (analyst$_{r}esult.json, synth_{r}esult.json, synth_{d}oc.md, critiques.json, discussions.json$ $, -(revisiontraceability) : , .$

Именно эта часть оценки проверяет вклад структурированной памяти. Если итоговый документ формально выглядит качественным, но невозможно понять, какие исследования поддерживают утверждения и какие поля аналитика были использованы, то ожидаемый эффект от памяти считается подтвержденным только частично. Напротив, конфигурация со структурированной памятью должна показывать более явную связь между корпусом источников, извлечением данных, критикой и синтезом.

2.5.5 Метрики цикличности и использования критики

Эта группа метрик применяется к конфигурации с циклами и к итоговой конфигурации с памятью и циклами. Цикл считается полезным только в том случае, если он исправляет конкретные проблемы, обнаруженные критиком, а не просто увеличивает число сообщений и повторных запусков.

Для каждого прогона фиксируются три показателя:

- число повторных попыток или циклов, - доля замечаний критика, которые были учтены в следующей попытке, - изменение качества результата после повторной попытки.

В задачах научных агентов такая оценка особенно важна, потому что современные тестовые наборы показывают падение качества при длинном горизонте действий. Например, SciAgentGym оценивает не только финальный успех, но и способность агента выполнять многошаговые научные процессы; авторы показывают, что при росте длины траектории качество инструментального выполнения заметно падает [2]. Для Clawd-Lab это означает, что цикличность должна оцениваться как управляемый механизм исправления ошибок, а не как бесконтрольное увеличение числа шагов.

2.5.6 LLM-судья (LLM-as-a-Judge)

LLM-судья используется как вторичный слой оценки и разделяется по ролям. Для аналитика применяется отдельный промпт судьи: он получает JSON с извлеченными данными и эталонную таблицу, после чего проверяет несовпадения, смешение исследований, неподтвержденные числа и пропуски. Для синтезатора применяется другой промпт судьи: он получает итоговый текст и результат аналитика, после чего ищет неподтвержденные утверждения, потерю ключевых выводов, слабые ограничения и ошибки в описании неопределенности.

Чтобы снизить субъективность LLM-оценки, используются следующие правила:

- дискретная шкала оценок с описанными уровнями, - несколько примеров хороших и плохих ответов, - запрет использовать внешние знания вне предоставленных артефактов, - перемешанный порядок артефактов перед оценкой, - температура, близкая к нулю, - скрытое или кратко суммированное рассуждение судьи, чтобы сохранить объяснение оценки без лишней генерации, - ручная проверка спорных и худших случаев.

Современные работы по LLM-судьям показывают, что такие оценки требуют калибровки на человеческой разметке и могут быть смещены из-за неоднозначности шкалы, предпочтения более длинных ответов или различий между интерпретациями рецензентов [6, 7]. Поэтому LLM-судья не заменяет эталонную таблицу и автоматические проверки, а помогает масштабировать качественный разбор и объяснить ошибки.

2.5.7 Метрики воспроизводимости и изоляции прогонов

Воспроизводимость в данной работе понимается как возможность восстановить ход выполнения задачи и получить сопоставимый результат при повторном запуске в тех же условиях. Для агентных систем это особенно сложно, поскольку результат зависит от модели, промпта, порядка сообщений, доступных инструментов, состояния памяти и истории лаборатории.

Оцениваются три показателя:

- полнота и восстановимость протокола запуска: сохранены конфигурация, идентификатор прогона (run id), идентификатор лаборатории (lab slug), идентификаторы агентов, модель, шаблоны промптов и задач, входные

данные, ограничения, параметры и время запуска, - изоляция прогонов: используются ли отдельные идентификаторы лабораторий и префиксы хранилища, - отсутствие утечек: не попадает ли в входные данные информация из эталонной таблицы или результатов других конфигураций.
В данной постановке важнее не широкий состязательный тестовый набор, а чистота и повторяемость одного клинического тестового набора. Поэтому устойчивость оценивается ограниченно: проверяется, что система не расширяет корпус статей, не использует внешние источники, не переносит память между независимыми лабораториями и не смешивает результаты разных конфигураций.

2.5.8 Метрики вычислительной эффективности

Поскольку предлагаемые доработки добавляют память и дополнительные попытки выполнения, они могут повышать качество ценой роста затрат. Поэтому вместе с качеством фиксируются три инженерные метрики:
- общее время прогона, - число повторных попыток или циклов, - число токенов или примерная стоимость выполнения
Эти метрики не являются основными целевыми метриками, но они нужны для честного анализа компромисса между качеством и затратами и для принятия решения по H_A. Например, если итоговая конфигурация улучшает качество на небольшую величину, но увеличивает стоимость в несколько раз, такой результат трудно считать практически полезным. Напротив, умеренное увеличение времени может быть приемлемым, если система становится заметно более воспроизводимой и лучше учитывает критику.

2.5.9 Критерии принятия гипотезы

Решение по гипотезам принимается относительно формулировок H_0 и H_A, заданных выше. Нулевая гипотеза H_0 не отклоняется, если конфигурации с памятью, с циклами и с обеими доработками не показывают устойчивого преимущества перед базовой конфигурацией или если наблюдаемое улучшение объясняется единичными примерами, ростом стоимости выполнения, изменением внешних условий эксперимента или утечкой информации между прогонами.
Нулевая гипотеза H_0 отклоняется в пользу альтернативной гипотезы H_A, если хотя бы одна улучшенная конфигурация показывает прирост по

основным метрикам без ухудшения других ключевых групп, а абляционное сравнение показывает, какой именно компонент дал вклад. Основными целевыми метриками являются:

- точность заполнения полей аналитиком; - верность синтеза относительно результата аналитика; - трассируемость утверждений к исследованиям и извлеченным полям.

Для количественных или порядковых оценок используется парное сравнение: один и тот же корпус статей и одни и те же шаблоны задач сравниваются в разных конфигурациях. Если проводится несколько прогонов, например r01, r02, r03, для оценки устойчивости результата применяются доверительные интервалы, бутстрап или непараметрические тесты для связанных выборок. Если число прогонов мало, вывод не должен опираться только на p-value, в этом случае основную роль играет качественная абляция, показывающая, какой именно компонент системы дал вклад.

Анализ проводится в несколько шагов. Сначала базовая конфигурация сравнивается с конфигурацией со структурированной памятью, чтобы оценить вклад памяти. Затем базовая конфигурация сравнивается с конфигурацией с циклами, чтобы оценить самостоятельный вклад формализованного повторного выполнения. После этого итоговая конфигурация с памятью и циклами сравнивается с обеими частичными конфигурациями. Такой дизайн позволяет не смешивать эффект памяти с эффектом повторных попыток и не приписывать улучшение случайному поведению агента-разведчика, главного исследователя или внешнего агента. Для отклонения H_0 недостаточно, чтобы итоговый отчет выглядел лучше в одном удачном примере. Нужно показать, что улучшение видно в конкретных артефактах: извлечение данных ближе к эталонной таблице, синтез верен результату аналитика, а утверждения прослеживаются к исследованиям. Если улучшается только читаемость текста, но не точность, трассируемость или верность синтеза, результат интерпретируется как частичное свидетельство в пользу H_A, но не как полное подтверждение архитектурной гипотезы.

2.5.10 Протокол проведения экспериментов

Эксперимент проводится по единому протоколу. Сначала выполняется этап 0: заморозка входов. На этом этапе готовятся фиксированный список PMID/PMCID, корпус статей, эталонная таблица, шаблоны задач и правила запрета на расширение корпуса. Эти материалы не создаются агентами во время сравнения и не считаются результатом какой-либо конфигурации.

Далее для каждой конфигурации создается отдельная лаборатория, отдельные идентификаторы агентов и отдельный идентификатор лаборатории (lab slug). На этапе 1 главный исследователь создает фиксированную задачу обзора литературы с инструкцией суммировать только исследования из точного списка источников. Эта задача нужна как вход для аналитика, а не как главная метрика. На этапе 2 главный исследователь создает фиксированную задачу анализа с жесткой схемой результата. На этапе 3 создается задача критики для проверки результата аналитика. На этапе 4 создается задача синтеза, которая использует только принятый результат аналитика и формирует документ в формате Markdown по заданной структуре.

После выполнения сохраняются все итоговые и промежуточные артефакты: результат аналитика, результат синтезатора, финальный документ в формате Markdown, критика, обсуждения, журнал активности и метаданные. Оценщик получает не только финальный документ, но и структурированный пакет выполнения. При экспертной или LLM-оценке порядок артефактов перемешивается, а название конфигурации по возможности скрывается.

Далее проводится оценка по трем уровням. На первом уровне автоматические проверки сравнивают результат аналитика с эталонной таблицей. На втором уровне проверяется синтез: структура документа, верность результату аналитика и неподтвержденные утверждения. На третьем уровне анализируются инженерные метрики и процесс: время, стоимость, число сообщений, число циклов и полнота выгрузки.

Для каждого типа ошибок формируется разбор: где система пропустила исследование, смешала исследования, неверно извлекла числовое значение, добавила неподтвержденное утверждение, не учла критику или ухудшила результат после повторной попытки. Такой разбор важен не меньше средних метрик, поскольку показывает границы применимости отдельных доработок.

Источники:

1. Agent Laboratory: Using LLM Agents as Research Assistants - https://arxiv.org/abs/2501.04227 2. SciAgentGym: Benchmarking Multi-Step Scientific Tool-use in LLM Agents - https://arxiv.org/abs/2602.12984 3. Holistic Evaluation of Language Models - https://arxiv.org/abs/2211.09110 4. Hugging Face LLM Evaluation Guidebook - https://github.com/huggingface/evaluation-guidebook 5. UpTrain predefined evaluations documentation - https://docs.uptrain.ai/predefined-evaluations/overview 6. Validating LLM-as-a-Judge Systems under Rating Indeterminacy - https://www.microsoft.com/en-us/research/publication/validating-llm-as-a-judge-systems-under-rating-indeterminacy/ 7. Human-anchored longitudinal comparison of generative AI with a bias-calibrated LLM-as-judge - [https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0339920](https://journ
8. Fisher R. A. The Design of Experiments. Oliver and Boyd, 1935.

2.6 Ожидаемый проверочный прототип

2.6.1 Целевая архитектура экспериментального прототипа

Ожидаемый проверочный прототип (Proof-of-Concept) в данной работе представляет собой не отдельную новую платформу, а экспериментальное расширение ClawdLab, предназначенное для проверки двух архитектурных доработок: структурированной памяти исследования и управляемой цикличности выполнения задач. Прототип должен сохранить текущую роль ClawdLab как слоя координации внешних агентов, но добавить к нему более явное представление исследовательского состояния.

На уровне целевой архитектуры прототип включает несколько основных компонентов.

Первый компонент - существующее рабочее пространство лаборатории. Лаборатория задает общий исследовательский контекст: научный вопрос, роли агентов, обсуждения, документы, артефакты и журналы активности. В

рамках эксперимента лаборатория рассматривается как контейнер для одного исследовательского направления, внутри которого запускаются отдельные задачи типов анализ и глубокое исследование.

Второй компонент - расширенная модель задачи. Сущность задачи (Task) сохраняет роль исследовательской цели: она описывает, что именно необходимо проанализировать, какую гипотезу проверить или какой исследовательский вопрос проработать. При этом задача не должна смешиваться с конкретным проходом выполнения. Для этого вводится отдельный уровень попыток выполнения.

Третий компонент - сущность попытки выполнения (TaskAttempt или TaskRun). Она фиксирует один конкретный проход решения задачи: номер попытки, план выполнения, использованные источники, связанные задания внешних провайдеров, созданные артефакты, результат, замечания критика, решение по итогам проверки и отличие от предыдущей попытки. Такой подход позволяет не менять базовый жизненный цикл задачи, но при этом сделать цикличность явной и машинно-обрабатываемой.

Четвертый компонент - структурированная память исследования. Для прототипа выбирается не полноценная графовая память с формальным пересмотром убеждений, а более простая многоуровневая модель, близкая к Hierarchical Cognitive Caching [1]. В ней можно выделить три уровня:

- рабочий уровень: текущая задача, активная попытка, последние действия агентов и ближайшие артефакты, - эпизодический уровень: история попыток, критика, ошибки, промежуточные выводы и причины повторных запусков,
- канонический уровень: принятые выводы, финальные документы, ограничения и устойчивые результаты исследования.

Пятый компонент - слой связей между артефактами. В базовой конфигурации ClawdLab уже хранит обсуждения, документы, задания внешних провайдеров и журналы активности, однако эти элементы не всегда образуют явную исследовательскую структуру. В прототипе должны фиксироваться связи вида: задача связана с гипотезой, попытка связана с задачей, результат связан с попыткой, критика связана с результатом, новая попытка связана с причиной пересмотра, а итоговый вывод связан с использованными источниками и ограничениями.

Таким образом, целевая архитектура проверочного прототипа может быть

представлена как расширение существующих сущностей ClawdLab, а не как их замена. Главное изменение состоит в том, что исследовательский процесс становится представлен не только через сообщения и документы, но и через структуру: задача, попытка выполнения, артефакты, критика, решения и уровни памяти.

2.6.2 Ожидаемые сценарии работы

Прототип должен поддерживать два основных экспериментальных сценария, соответствующих выбранным типам задач ClawdLab.

Первый сценарий - задача анализа. Пользователь или главный исследователь создает лабораторию для определенного научного вопроса и внутри нее формулирует задачу типа анализ. Для задачи задаются исходная гипотеза, набор источников, ожидаемый формат результата и ограничения выполнения. Система запускает первую попытку, в рамках которой агент анализирует материалы, выделяет аргументы, формирует промежуточные выводы и создает аналитический артефакт.

После выполнения попытки результат передается на критику. Критик или главный исследователь фиксирует замечания: например, неполный учет источников, противоречие в аргументации, слабую связь между выводом и доказательствами или отсутствие ограничений. Если результат признается достаточным, он переходит в канонический уровень памяти как принятый вывод. Если результат требует доработки, создается новая попытка, связанная с предыдущей и с причиной пересмотра.

Второй сценарий - задача глубокого исследования. В этом случае входная постановка может быть шире: не только проверить одну гипотезу, но и проработать исследовательский вопрос, выделить возможные направления проверки, сопоставить подходы и подготовить более развернутый вывод. Такой сценарий требует более длинной траектории: несколько промежуточных выводов, критику, уточнение исходной постановки и итоговый синтез.

Для глубокого исследования особенно важна связка памяти и цикличности. Структурированная память должна сохранять, какие подзадачи были выделены, какие источники использованы, какие промежуточные выводы были получены и какие замечания появились после критики. Циклическая модель должна позволять вернуться к предыдущей попытке неформально

не ”с нуля”, а как к документированному этапу исследования: с указанием причины, измененного плана и ожидаемого улучшения.

Ожидаемый результат работы прототипа включает не только финальный текстовый документ, но и пакет исследовательских артефактов:

- карточку задачи, - список исходных источников, - одну или несколько попыток выполнения, - результаты заданий внешних провайдеров, - промежуточные выводы, - замечания критика, - решения о принятии результата или повторном запуске, - итоговый отчет, - связи между задачей, попытками, критикой, источниками и выводами.

Именно этот пакет артефактов используется в дальнейшем для оценки качества, воспроизводимости и вклада архитектурных доработок.

2.6.3 Возможности дальнейшего развития прототипа

Ожидаемый прототип имеет несколько принципиальных ограничений и возможностей дальнейшего развития.

Он не должен реализовывать полный автономный цикл научного открытия. Его задача состоит в проверке двух конкретных механизмов внутри Clawd-Lab: структурированной памяти и циклического выполнения. Поэтому прототип ограничивается задачами анализа и глубокого исследования и не претендует на автоматизацию всех типов научной деятельности.

В прототип не включается динамический выбор языковых моделей. Clawd-Lab работает с внешними агентами, которые имеют собственные модели и исполнительные контуры. Платформа может передавать агентам контекст, роль, задачу и артефакты, но не управляет напрямую тем, какую модель внешний агент использует внутри своей исполнительной среды. Поэтому выбор моделей фиксируется как условие эксперимента, а не как часть разрабатываемой архитектуры.

Также в рамках проверочного прототипа не реализуется полноценная графовая память (graph-native memory) с формальной семантикой поддержки, опровержения и пересмотра убеждений [2]. Такой подход является перспективным, но для первого прототипа он избыточен: он потребовал бы отдельной онтологии, правил версионирования и более сложного графового хранилища. В данной работе используется более легкая артефактно-ориентированная память с элементами иерархического кэширования.

Карта знаний (mind-map) или графовая визуализация знания рассматривается как возможное развитие, но не как обязательная часть экспериментального прототипа [3]. В проверочном прототипе достаточно зафиксировать машинно-обрабатываемые связи между задачами, попытками, источниками, критикой и выводами. Визуальное представление этих связей может быть добавлено позже как интерфейсный слой.

Также цикличность ограничивается заранее заданными правилами остановки: максимальным числом попыток, решением главного исследователя, отсутствием улучшения или превышением вычислительного бюджета. Это важно, поскольку бесконтрольные повторные запуски могут повышать стоимость без реального улучшения результата.

Источники:

1. Toward Ultra-Long-Horizon Agentic Science: Cognitive Accumulation for Machine Learning Engineering - https://arxiv.org/abs/2601.10402
2. Graph-Native Cognitive Memory for AI Agents - https://arxiv.org/abs/2603.17244
3. MindMap: Knowledge Graph Prompting Sparks Graph of Thoughts in Multi-Hop Reasoning - https://arxiv.org/abs/2308.09729 4. Towards an AI Co-Scientist - https://arxiv.org/abs/2502.18864
5. Agent Laboratory: Using LLM Agents as Research Assistants - https://arxiv.org/abs/2501.04227

3 ОПИСАНИЕ ПРЕДЛАГАЕМОГО РЕШЕНИЯ

3.1 Архитектура продакшн-решения и System Design

В первых двух главах были рассмотрены предметная область, архитектурные ограничения исходной системы и дизайн экспериментальной проверки предлагаемых доработок. В третьей главе внимание переносится с уровня проектирования на уровень реализации: описывается целевая архитектура продакшн-решения, принципы интеграции ClawdLab и OpenClaw, особенности развертывания на собственной инфраструктуре и подходы к сопровождению системы.

С практической точки зрения разрабатываемое решение представляет собой self-hosted агентную среду для исследовательских задач, развернутую на VPS-сервере под управлением Ubuntu. В качестве технологической основы используются OpenClaw как инфраструктурный слой подключения и исполнения агентов и ClawdLab как лабораторная надстройка, организующая исследовательский процесс, роли агентов и выполнение задач. Такое разделение позволяет отдельно рассматривать коммуникационный и интеграционный слой, прикладную исследовательскую логику, а также механизмы памяти, цикличности и воспроизводимости.

3.1.1 Цели и требования к целевой архитектуре

В контексте данной работы целевая архитектура должна удовлетворять нескольким требованиям. Она должна быть воспроизводимой и управляемой при развертывании на выделенном сервере без зависимости от внешней облачной платформы оркестрации. И она должна поддерживать независимый запуск нескольких агентных контуров, поскольку OpenClaw и ClawdLab выполняют разные функции и могут обслуживаться раздельно. Также архитектура должна допускать локальные доработки исходного кода, необходимые для адаптации системы под исследовательский сценарий.

Дополнительно к этим требованиям предъявляются требования к прозрачности выполнения задач, расширяемости провайдеров данных, контролю над конфигурацией окружения и удобству сопровождения. Поскольку работа ориентирована не только на запуск MVP, но и на

дальнейшее развитие решения, архитектура должна допускать поэтапное внедрение доработок, в том числе механизмов структурированной памяти и управляемой цикличности исполнения.

3.1.2 Логическая структура решения

На логическом уровне систему можно представить как набор взаимосвязанных слоев. Нижний слой образует серверная инфраструктура: VPS-сервер, операционная система Ubuntu, сетевые настройки, переменные окружения и управляющие сессии. Поверх него располагается слой исполнения приложений, в котором OpenClaw и ClawdLab запускаются как два отдельных процесса в разных tmux-сессиях. Такое решение упрощает изоляцию, перезапуск и наблюдение за журналами каждого компонента.

Следующий слой составляет прикладная логика ClawdLab: маршруты API, постановка и запуск задач, взаимодействие с провайдерами, обработка результатов и формирование исследовательских артефактов.

Над этим слоем располагаются механизмы исследовательского управления, включающие память, фиксацию артефактов, логику повторных попыток и, в перспективе, управляемую цикличность.

3.1.3 Физическое развертывание на VPS-сервере

Физическая схема развертывания в текущей реализации является минималистичной и ориентированной на MVP. Все основные компоненты размещаются на одном VPS-сервере под управлением Ubuntu. OpenClaw и ClawdLab запускаются вручную в отдельных tmux-сессиях, что позволяет независимо управлять жизненным циклом каждого сервиса, просматривать их вывод и при необходимости быстро восстанавливать рабочее состояние после сбоев или обновлений.

Такой способ развертывания не является финальным, но он соответствует этапу прототипирования. Он обеспечивает достаточный уровень контроля над окружением, прозрачность исполнения и низкий порог внесения изменений в кодовую базу. В дальнейшем эта схема может быть расширена до использования systemd, контейнеризации или внешнего reverse proxy, но на текущем этапе ручное управление через tmux лучше согласуется с задачами отладки и быстрой итерации.

3.2 Разработка сервиса и MVP-интеграция

В рамках MVP основное внимание уделяется не полному перепроектированию ClawdLab, а обеспечению работоспособного исследовательского контура, который можно развернуть на собственном сервере, настроить через переменные окружения и использовать для выполнения задач обзора литературы и анализа.

3.2.1 Базовый контур MVP

3.2.2 Доработка literature provider

В ходе запуска сервиса было выявлено, что система не поддерживает подключение к провайдерам литературы напрямую, для этого нужна надстройка, которая позволила бы агентам правильно обращаться в источниики данных. Для этого в файл lib/providers.ts была добавлена логика работы с источниками, предлагаемыми авторами clawdlab (пока что только europe pmc). Эта доработка позволяет использовать Europe PMC fallback как внутренний источник литературы.

Вторым изменением стало обновление маршрута запуска literature job: логика маршрута была адаптирована так, чтобы задача могла завершаться сразу, без необходимости ожидания внешнего асинхронного провайдера. Тем самым был упрощен путь исполнения запроса в конфигурации, где используется встроенный провайдер Europe PMC.

С инженерной точки зрения это изменение уменьшает связность между стартом задачи и внешним асинхронным контуром выполнения. Для MVP это особенно полезно, поскольку снижает сложность интеграции, упрощает отладку и уменьшает число состояний, которые необходимо учитывать при сопровождении системы. Одновременно такое изменение формирует более чистую основу для дальнейшего введения нескольких режимов исполнения: синхронного, асинхронного и гибридного.

Чтобы сделанные доработки могли использоваться не только в локальной рабочей копии, но и в повторяемом процессе развертывания, были обновлены конфигурационные и пользовательские документы. Новые переменные окружения были задокументированы в .env.example и README.md. Это позволяет явно зафиксировать поддерживаемый режим запуска literature provider и сократить число скрытых зависимостей при

переносе решения на другой сервер или при повторной установке.
Фактически это означает, что сценарий получения литературы может быть активирован без отдельного внешнего literature agent API.

3.2.3 Архитектура модуля структурированной памяти

В данной работе внедряется следующий вариант HCC-памяти, ориентированный на запуск слоя исследовательской памяти. В отличие от системы ML Masters 2.0 в этой версии не вводятся отдельный слой снимков контекста (context snapshot), графовая модель связей, поля sourceVersion и интервалы актуальности validFrom/validTo. Такое ограничение уменьшает сложность схемы данных и логики сопровождения, но при этом сохраняет главный архитектурный принцип HCC: разделение памяти по уровням зрелости и роли в исследовательском процессе.
В основе модуля лежат три сущности: MemoryRecord, MemorySourceRef и MemoryPromotionJob. Вместе они образуют минимально достаточную архитектуру для хранения самих записей памяти, фиксации происхождения знаний и наблюдаемого продвижения памяти между уровнями. Тем самым достигается полноценная трехуровневая модель, включающая рабочую, эпизодическую и каноническую память, но без преждевременного усложнения модели графами и версиями контекста.
MemoryRecord выступает главной сущностью памяти. Эта таблица хранит принадлежность записи лаборатории и, при необходимости, конкретной задаче или состоянию лаборатории, уровень памяти (working, episodic, canonical), тип записи, статус, текстовое содержимое, а также дополнительные структурированные данные. Для поддержки практического использования в записи также фиксируются уверенность, важность, источник происхождения, авторство и связи эволюции памяти. Последнее реализуется через три типа lineage-ссылок: parentMemoryId для иерархического родительства, promotedFromId для указания записи, из которой текущая память была поднята на следующий уровень, и supersedesId для замещения более старой версии знания новой.
Такое устройство MemoryRecord позволяет реализовать HCC-логику даже без графовой модели. Поле уровня задает архитектурное разделение памяти, тип записи нужен для извлечения релевантного контекста и фильтрации, а поля sourceType/sourceId и lineage-отношения обеспечивают происхождение

данных (provenance) и прослеживаемость эволюции знания. Иначе говоря, система получает возможность не только хранить вывод, но и понимать, откуда он возник, к какому эпизоду относится и какую запись он уточняет или заменяет.

MemorySourceRef используется как дополнительный слой ссылок на источники. Он нужен потому, что одна запись памяти в реальном исследовательском процессе нередко опирается сразу на несколько сущностей. Например, итоговая запись типа canonical/finding может быть выведена не из одного объекта, а из комбинации задачи, голосования, документа, результата задания внешнего провайдера и лабораторного состояния. Поэтому вместо жесткой графовой модели вводится нормализованный слой происхождения данных (provenance), в котором для каждой записи памяти можно указать тип источника, его идентификатор, краткий ярлык, фрагмент источника, указатель на место в источнике и произвольные метаданные.

MemoryPromotionJob представляет собой журнал дистилляции и продвижения памяти между уровнями. Он хранит, в какой лаборатории и для какой задачи или состояния запускалось продвижение, из какого уровня в какой оно выполнялось, какой триггер его вызвал, какие записи были использованы на входе, какие были созданы на выходе, а также итоговый статус, краткую сводку и возможную ошибку. Наличие такой сущности принципиально важно, поскольку без нее HCC-продвижение превращается в скрытую внутреннюю логику. С журналом же процесс дистилляции становится наблюдаемым, воспроизводимым и пригодным для отладки.

На уровне логики памяти первый релиз поддерживает три слоя. Рабочая память играет роль краткосрочного рабочего контекста. В нее попадают текущий контекст задачи, активный план, свежие результаты провайдеров, временные заметки по исследовательской ветке и недавние артефакты, которые еще не прошли осмысление. Для этого уровня характерны быстрое обновление, допустимость шума и незавершенности, а также возможность истечения по expiresAt.

Эпизодическая память фиксирует завершенные или частично завершенные эпизоды работы. Сюда относятся краткие итоги попыток, локальные ошибки и обходные решения, сводки критики, сводки обсуждений, извлеченные

уроки по отдельным эпизодам и промежуточные выводы, которые еще не должны считаться каноном. По сравнению с рабочим слоем этот слой уже сжат и структурирован, но все еще может содержать конкурирующие или конфликтующие версии интерпретации результатов.

Каноническая память хранит устойчивое знание лаборатории. На этот уровень поднимаются принятые выводы, сводки состояния лаборатории, финальные выводы, устойчивые правила лаборатории и межзадачные уроки, которые действительно подтверждены. В отличие от рабочей и эпизодической памяти, канонический слой изменяется редко, обычно не истекает по времени и обновляется не перезаписью, а через создание новой записи с указанием supersedesId. Это важно для сохранения истории эволюции знаний и предотвращения потери старых, но когда-то значимых выводов.

Рабочие записи создаются, когда задача взята в работу, запущено или завершено задание внешнего провайдера, появился новый активный план либо был загружен новый набор данных или артефакт. Эпизодические записи создаются, когда задача завершена, появляется критика, ветка обсуждения дошла до состояния, пригодного для сводки, или в одной ветке накопилось достаточно активности. Канонические записи создаются тогда, когда задача перешла в статус принятой, состояние лаборатории активировано или завершено, итоговый документ признан финальным либо PI или задание консолидации поднимает устойчивый вывод.

Базовый пайплайн продвижения памяти между уровнями строится как трехшаговая схема. Первый шаг - переход из рабочей памяти в эпизодическую память. Он выполняется тогда, когда активная работа завершилась и из набора сырых результатов уже можно собрать короткий осмысленный эпизод. Второй шаг - переход из эпизодической памяти в каноническую память. Он выполняется, когда результат прошел через принятие, голосование или решение о состоянии, вывод повторяется и подтверждается или вошел в лабораторные документы и устойчивые выводы. Третий шаг - замещение внутри канонической памяти. Он нужен в тех случаях, когда старый вывод уточняется, новая версия надежнее и предыдущая запись больше не должна считаться актуальной. Тогда новая запись не затирает старую, а получает ссылку supersedesId на прежний find-

ing.

Чтение памяти также строится по правилам извлечения релевантного контекста. Для активной задачи сначала извлекаются рабочие записи по taskId, затем эпизодические записи по той же задаче, а затем релевантные канонические записи по labId и нужным типам памяти.

С точки зрения интеграции с текущей моделью ClawdLab новая память не заменяет существующие сущности, а располагается поверх них. Task и ProviderJob поставляют данные для рабочего слоя, TaskCritique, LabDiscussion и LabActivityLog образуют основу эпизодического уровня, а принятая задача, LabState и LabDocument становятся естественными источниками канонических знаний. Тем самым архитектура памяти остается согласованной с уже существующей доменной моделью и не требует разрушительного переписывания текущего приложения.

Следующим шагом после фиксации этой схемы является описание API, т. е. маршрутов вида POST /api/labs/slug/memory, GET /api/labs/slug/memory, и т. д. Параллельно с этим определяютсяавтоматические триггеры запуска продвижения: после завершения задачи, после перехода голосования в статус accepted/rejected, после финализации документа и после завершения состояния лаборатории. Именно такое поэтапное введение API и триггеров делает HCC-память полезной доработкой для данной системы.Ы

3.2.4 Архитектура доработки цикличности исполнения

В текущей логике ClawdLab элементы цикла уже возможны: агент или PI может создать новую задачу, оставить критику, вернуться к обсуждению или попросить доработать результат. Однако такая цикличность остается неформальной: система не фиксирует, почему был начат новый проход, какая ошибка исправляется, чем новая попытка отличается от предыдущей и при каком условии цикл должен завершиться. Поэтому архитектурно предлагается отделить задачу как цель исследования от попытки выполнения как конкретного прохода решения. Для этого может быть введена сущность TaskAttempt или TaskRun, связанная с исходной задачей, результатом критики, использованными артефактами, статусом попытки и причиной повторного запуска.

Фиксированные правила цикличности могут быть описаны через отдельную политику выполнения для типа задачи или конкретной лаборатории.

Такая политика задает допустимые триггеры повтора: наличие критики со статусом ”нужно исправить”, обнаружение пропущенных источников, неподтвержденные числовые значения, противоречие между полями результата или недостаточная трассируемость вывода к источникам. В этой же политике задаются условия остановки: максимальное число попыток, отсутствие улучшения по выбранным метрикам, превышение бюджета времени или стоимости, а также ручное решение PI принять текущий результат. Тогда цикл выполнения становится не скрытым поведением агента, а наблюдаемым конечным процессом: задача порождает попытку, попытка получает результат и критику, правило принимает решение о завершении или повторе, а новая попытка наследует только релевантный контекст и явно указывает, какую проблему она должна исправить.

С точки зрения реализации этот слой встроен в существующую серверную логику ClawdLab без изменения роли внешних агентов. Сервер хранит состояние цикла, связывает TaskAttempt с Task, TaskCritique, MemoryRecord и артефактами, а внешний агент получает уже подготовленный контекст следующей попытки. API включает операции создания попытки, завершения попытки, регистрации критики и принятия решения о следующем шаге. Такой подход сохраняет ClawdLab как координационный слой и одновременно делает цикличность проверяемой, воспроизводимой и пригодной для последующего сравнения в эксперименте.

3.3 Развертывание, DevOps и MLOps-практики

3.3.1 Организация развертывания и запуск сервисов

3.4 Тестирование и верификация решения

3.4.1 План тестирования и валидации

4 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА

4.1 Реализация доработки со структурированной памятью

В рамках практической части работы была реализована доработка ClawdLab, связанная со структурированной памятью исследования. Архитектурно память реализована как отдельный слой поверх существующих сущностей ClawdLab. В реализованной схеме используются три уровня памяти. Рабочая память хранит текущий контекст выполнения: активную задачу, план, свежие результаты провайдеров и временные заметки. Эпизодическая память фиксирует завершенные фрагменты работы: краткие итоги попыток, обсуждений, критики и локальные уроки, полученные в ходе выполнения задачи. Каноническая память содержит устойчивые выводы лаборатории: принятые результаты, финальные заключения, правила лаборатории и обобщения, которые могут использоваться в следующих задачах. Такое разделение позволяет постепенно переводить информацию от сырого рабочего контекста к более устойчивому знанию.

Также была заложена логика продвижения памяти между уровнями. Для этого используется журнал операций продвижения, который фиксирует, из какого уровня в какой выполнялась дистилляция, какие записи были использованы на входе, какие записи были созданы на выходе и чем завершилась операция. Благодаря этому процесс обновления памяти становится наблюдаемым и воспроизводимым: можно понять, почему конкретный вывод оказался в каноническом слое и на какие данные он опирается.

4.2 Итоговое сравнение метрик между конфигурациями

ЗАКЛЮЧЕНИЕ

В заключении подведены итоги проведенного исследования.

Основные выводы работы:

– -

– -

– -

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Иванов И. И., Петров П. П. Ансамблевые методы повышения точности классификации данных // Журнал машинного обучения. — 2019. — Vol. 12, no. 3. — P. 15–28.